```
import { lazy, Suspense } from 'react'
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import { AuthProvider } from './providers/AuthProvider'
import Layout from './components/Layout'
import ProtectedRoute from './components/ProtectedRoute'
import { Spinner } from '@/components/ui/Spinner'

const HomePage = lazy(() => import('./pages/HomePage'))
const LoginPage = lazy(() => import('./pages/LoginPage'))
const GardenPage = lazy(() => import('./pages/GardenPage'))
const CheckinPage = lazy(() => import('./pages/CheckinPage'))
const BadgePage = lazy(() => import('./pages/BadgePage'))
const ReportPage = lazy(() => import('./pages/ReportPage'))
const SettingsPage = lazy(() => import('./pages/SettingsPage'))
const StoolPage = lazy(() => import('./pages/StoolPage'))
const ClassroomPage = lazy(() => import('./pages/ClassroomPage'))
const ProfilePage = lazy(() => import('./pages/ProfilePage'))

function PageLoader() {
  return (
    <div className="flex items-center justify-center h-full">
      <Spinner />
    </div>
  )
}

export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Suspense fallback={<PageLoader />}>
          <Routes>
            <Route path="/login" element={<LoginPage />} />
            <Route element={<Layout />}>
              <Route element={<ProtectedRoute />}>
                <Route path="/" element={<HomePage />} />
                <Route path="/garden" element={<GardenPage />} />
                <Route path="/checkin" element={<CheckinPage />} />
                <Route path="/stool" element={<StoolPage />} />
                <Route path="/classroom" element={<ClassroomPage />} />
                <Route path="/badges" element={<BadgePage />} />
                <Route path="/report" element={<ReportPage />} />
                <Route path="/profile" element={<ProfilePage />} />
                <Route path="/settings" element={<SettingsPage />} />
              </Route>
            </Route>
          </Routes>
        </Suspense>
      </AuthProvider>
    </BrowserRouter>
```

```
  )
}
import { useEffect, useRef, useState, type CSSProperties } from 'react'
import { UiIcon } from '@/lib/uiIcons'
import type { BadgeDef, BadgeRarity } from '@/types/badges'

const BADGE_FRAMES: Record<BadgeRarity, string> = {
  bronze: '/assets/badges/frames/ui_badge_frame_bronze.webp',
  silver: '/assets/badges/frames/ui_badge_frame_silver.webp',
  gold: '/assets/badges/frames/ui_badge_frame_gold.webp',
}

const PAGE_W = 300
const PAGE_H = 190
const SPINE_W = 18
// 书本布局基准尺寸（含封面外延 / 阴影空间），用于按容器等比缩放
const BOOK_W = 618
const BOOK_H = 208

// 每页 4 枚徽章，左页 + 右页 = 一次翻开看到 8 枚
const PER_PAGE = 4
const PER_SPREAD = 8
const FLIP_MS = 520
const FLIP_HALF = 250

interface BadgeBookProps {
  defs: BadgeDef[]
  awardedCodes: Set<string>
  awardedRarities?: Record<string, BadgeRarity>
}

function LockBadge({ size = 10 }: { size?: number }) {
  return (
    <div className="absolute -bottom-[2px] -right-[2px] w-[16px] h-[16px] rounded-full
bg-[#455A64] border border-white/70 flex items-center justify-center shadow-[0_1px_3px_
rgba(0,0,0,0.3)]">
      <UiIcon name="lock" size={size} className="text-white" />
    </div>
  )
}

function BadgeCircle({ def, awarded, rarity }: { def: BadgeDef; awarded: boolean; rarit
y?: BadgeRarity }) {
  return (
    <div className={`relative ${awarded ? 'drop-shadow-[0_0_6px_rgba(255,205,0,0.55)]'
: 'grayscale brightness-[0.75] opacity-60'}`}>
      <img src={def.icon_url} alt={def.name} className="w-[80px] h-[80px] object-contai
n" />
      {awarded && rarity && (
        <img
          src={BADGE_FRAMES[rarity]}
          alt=""
          className="absolute left-1/2 top-1/2 -translate-x-1/2 -translate-y-1/2 w-[92p
x] h-[92px] object-contain pointer-events-none"
        />
      )}
```

```
      {!awarded && <LockBadge />}
    </div>
  )
}

interface FaceProps {
  side: 'left' | 'right'
  defs: BadgeDef[]
  awardedCodes: Set<string>
  awardedRarities?: Record<string, BadgeRarity>
  style?: CSSProperties
}

function Face({ side, defs, awardedCodes, awardedRarities, style }: FaceProps) {
  const round = side === 'left' ? 'rounded-l-[7px]' : 'rounded-r-[7px]'
  const bg = side === 'left'
    ? 'radial-gradient(ellipse at 35% 18%, #FFFDF7 25%, #F3E7C4 100%)'
    : 'radial-gradient(ellipse at 65% 18%, #FFFDF7 25%, #F3E7C4 100%)'
  return (
    <div
      className={`absolute inset-0 ${round} overflow-hidden`}
      style={{ backfaceVisibility: 'hidden', background: bg, ...style }}
    >
      {side === 'left' ? (
        <>
          <div className="absolute left-0 top-0 bottom-0 w-[16px] bg-gradient-to-r from
-[#3D2B1F]/12 to-transparent" />
          <div className="absolute right-0 top-0 bottom-0 w-[4px] bg-gradient-to-l from
-[#B08A45]/25 to-transparent" />
        </>
      ) : (
        <>
          <div className="absolute right-0 top-0 bottom-0 w-[16px] bg-gradient-to-l fro
m-[#3D2B1F]/12 to-transparent" />
          <div className="absolute left-0 top-0 bottom-0 w-[4px] bg-gradient-to-r from-
[#B08A45]/25 to-transparent" />
        </>
      )}
      <div className="relative z-10 h-full grid grid-cols-2 grid-rows-2 gap-2 place-con
tent-center place-items-center px-4 py-1">
        {defs.map((def) => <BadgeCircle key={def.code} def={def} awarded={awardedCodes.
has(def.code)} rarity={awardedRarities?.[def.code]} />)}
      </div>
    </div>
  )
}

export default function BadgeBook({ defs, awardedCodes, awardedRarities }: BadgeBookPro
ps) {
  const rootRef = useRef<HTMLDivElement>(null)
  const flipRef = useRef<HTMLDivElement>(null)
  const busyRef = useRef(false)
  const timersRef = useRef<number[]>([])
  const [scale, setScale] = useState(1)

  const maxPage = Math.max(0, Math.ceil(defs.length / PER_SPREAD) - 1)
  const [page, setPage] = useState(0)
```

```
  const [leftDefs, setLeftDefs] = useState<BadgeDef[]>(() => defs.slice(0, PER_PAGE))
  const [frontDefs, setFrontDefs] = useState<BadgeDef[]>(() => defs.slice(PER_PAGE, PER
_SPREAD))
  const [backDefs, setBackDefs] = useState<BadgeDef[]>([])

  const leftSliceAt = (p: number) => defs.slice(p * PER_SPREAD, p * PER_SPREAD + PER_PA
GE)
  const rightSliceAt = (p: number) => defs.slice(p * PER_SPREAD + PER_PAGE, p * PER_SPR
EAD + PER_SPREAD)

  // 跟随容器尺寸等比缩放，使编辑模式下拖拽块大小可改变书本大小
  useEffect(() => {
    const el = rootRef.current
    if (!el) return
    const update = () => {
      const w = el.clientWidth
      const h = el.clientHeight
      if (w > 0 && h > 0) {
        const s = Math.min(w / BOOK_W, h / BOOK_H)
        setScale(Math.max(0.4, Math.min(1.6, s)))
      }
    }
    update()
    const ro = new ResizeObserver(update)
    ro.observe(el)
    return () => ro.disconnect()
  }, [])

  // defs 变化（新徽章解锁）时，把当前页收敛到合法范围并同步两页内容
  useEffect(() => {
    const m = Math.max(0, Math.ceil(defs.length / PER_SPREAD) - 1)
    setPage((p) => Math.min(p, m))
  }, [defs])

  useEffect(() => {
    setLeftDefs(leftSliceAt(page))
    setFrontDefs(rightSliceAt(page))
  }, [defs, page])

  useEffect(() => () => timersRef.current.forEach((t) => window.clearTimeout(t)), [])

  const flipTo = (target: number) => {
    if (busyRef.current || !flipRef.current) return
    busyRef.current = true
    const dir = target > page ? -1 : 1
    const el = flipRef.current
    setBackDefs(rightSliceAt(target))
    el.style.transition = `transform ${FLIP_MS}ms cubic-bezier(0.4,0.2,0.2,1)`
    el.style.transform = `translateZ(6px) rotateY(${dir * 180}deg)`
    timersRef.current.push(window.setTimeout(() => {
      setLeftDefs(leftSliceAt(target))
      setFrontDefs(rightSliceAt(target))
    }, FLIP_HALF))
```

```
    timersRef.current.push(window.setTimeout(() => {
      el.style.transition = 'none'
      el.style.transform = 'rotateY(0deg)'
      setPage(target)
      busyRef.current = false
    }, FLIP_MS))
  }

  const goNext = () => {
    if (page >= maxPage) return
    flipTo(page + 1)
  }
  const goPrev = () => {
    if (page <= 0) return
    flipTo(page - 1)
  }

  const handleShare = () => {
    const text = `我在肠道花园收集了 ${awardedCodes.size}/${defs.length} 枚徽章！`
    if (typeof navigator !== 'undefined' && navigator.share) {
      navigator.share({ text }).catch(() => {})
    } else if (typeof navigator !== 'undefined' && navigator.clipboard) {
      navigator.clipboard.writeText(text).catch(() => {})
    }
  }

  return (
    <div ref={rootRef} className="w-full h-full relative overflow-visible">
      {/* 书本整体按容器等比缩放，保持居中 */}
      <div
        className="absolute left-1/2 top-1/2"
        style={{
          width: BOOK_W,
          height: BOOK_H,
          transform: `translate(-50%,-50%) scale(${scale})`,
          transformOrigin: 'center',
        }}
      >
        <div
          className="relative flex items-center justify-center"
          style={{ perspective: '1000px', perspectiveOrigin: '50% 24%' }}
        >
          {/* 整本书后仰倾斜 */}
          <div style={{ transformStyle: 'preserve-3d', transform: 'rotateX(26deg)' }}>
            <div className="relative flex items-start" style={{ transformStyle: 'preser
ve-3d' }}>
              {/* ── 左页：4 枚徽章 ── */}
              <div
                className="relative"
                style={{
                  width: PAGE_W,
```

```
                  height: PAGE_H,
                  transformStyle: 'preserve-3d',
                  transform: 'rotateY(11deg)',
                  transformOrigin: 'right center',
                }}
              >
                <div className="absolute -top-[4px] -left-[5px] -right-[2px] -bottom-[4
px] rounded-[9px] bg-[#EADFBC] shadow-[inset_2px_2px_2px_rgba(0,0,0,0.05)]" />
                <div className="absolute -top-[7px] -left-[8px] -right-[4px] -bottom-[7
px] rounded-[10px] bg-[#DCCFA6]" />
                <div className="absolute -top-[9px] -left-[10px] -right-[2px] -bottom-
[5px] rounded-[11px] bg-gradient-to-b from-[#5DA94F] via-[#3F8A38] to-[#2C7230] shadow-
[inset_0_2px_3px_rgba(255,255,255,0.22),inset_0_-2px_3px_rgba(0,0,0,0.25)]" />
                <div className="absolute -top-[9px] -left-[10px] -right-[2px] -bottom-
[5px] rounded-[11px] border-2 border-[#E9C767]/70" />
                <Face side="left" defs={leftDefs} awardedCodes={awardedCodes} awardedRa
rities={awardedRarities} />
              </div>

              {/* ── 书脊 ── */}
              <div className="relative" style={{ width: SPINE_W, height: PAGE_H, transf
orm: 'translateZ(2px)' }}>
                <div
                  className="absolute -inset-y-[6px] -left-[3px] -right-[3px]"
                  style={{
                    background: 'linear-gradient(90deg,#5C4128,#8B6B4A 40%,#5C4128 70%,
#4A331F)',
                    borderRadius: 5,
                    boxShadow:
                      'inset 0 2px 3px rgba(255,255,255,0.15), inset 0 -3px 4px rgba(0,
0,0,0.35), inset 2px 0 3px rgba(255,255,255,0.06)',
                  }}
                >
                  <div className="absolute left-1/2 -translate-x-1/2 top-[6px] bottom-
[6px] flex flex-col justify-between">
                    {Array.from({ length: 7 }).map((_, i) => (
                      <span key={i} className="w-[3px] h-[3px] rounded-full bg-[#3B2A1
8]/60" />
                    ))}
                  </div>
                </div>
              </div>

              {/* ── 右页：4 枚徽章（可翻页） ── */}
              <div
                className="relative"
                style={{
                  width: PAGE_W,
                  height: PAGE_H,
                  transformStyle: 'preserve-3d',
                  transform: 'rotateY(-11deg)',
                  transformOrigin: 'left center',
                }}
              >
                <div className="absolute -top-[4px] -left-[2px] -right-[5px] -bottom-[4
px] rounded-[9px] bg-[#EADFBC] shadow-[inset_-2px_2px_2px_rgba(0,0,0,0.05)]" />
                <div className="absolute -top-[7px] -left-[4px] -right-[8px] -bottom-[7
px] rounded-[10px] bg-[#DCCFA6]" />
                <div className="absolute -top-[9px] -left-[2px] -right-[10px] -bottom-
[5px] rounded-[11px] bg-gradient-to-b from-[#5DA94F] via-[#3F8A38] to-[#2C7230] shadow-
[inset_0_2px_3px_rgba(255,255,255,0.22),inset_0_-2px_3px_rgba(0,0,0,0.25)]" />
                <div className="absolute -top-[9px] -left-[2px] -right-[10px] -bottom-
[5px] rounded-[11px] border-2 border-[#E9C767]/70" />
                {/* 翻页容器：围绕书脊（左缘）旋转 */}
                <div
                  ref={flipRef}
```

```
                  className="absolute inset-0"
                  style={{ transformStyle: 'preserve-3d', transformOrigin: 'left cente
r', transform: 'rotateY(0deg)' }}
                >
                  <Face side="right" defs={frontDefs} awardedCodes={awardedCodes} award
edRarities={awardedRarities} />
                  <Face side="right" defs={backDefs} awardedCodes={awardedCodes} awarde
dRarities={awardedRarities} style={{ transform: 'rotateY(180deg)' }} />
                </div>
                {/* 翻页控件（图标，不随纸面翻转） */}
                <div className="absolute bottom-[5px] left-1/2 -translate-x-1/2 z-20 fl
ex items-center gap-3 select-none">
                  <button
                    type="button"
                    onClick={goPrev}
                    className={`w-[20px] h-[20px] rounded-full flex items-center justif
y-center bg-[#8B6B4A]/15 text-[#7A5C3A] hover:bg-[#8B6B4A]/30 active:scale-90 transitio
n-all ${page > 0 ? '' : 'opacity-30 pointer-events-none'}`}
                  >
                    <UiIcon name="chevronLeft" size={12} />
                  </button>
                  <button
                    type="button"
                    onClick={goNext}
                    className={`w-[20px] h-[20px] rounded-full flex items-center justif
y-center bg-[#8B6B4A]/15 text-[#7A5C3A] hover:bg-[#8B6B4A]/30 active:scale-90 transitio
n-all ${page < maxPage ? '' : 'opacity-30 pointer-events-none'}`}
                  >
                    <UiIcon name="chevronRight" size={12} />
                  </button>
                </div>
              </div>
            </div>
          </div>

          {/* 底部投影 */}
          <div className="absolute -bottom-[20px] left-1/2 -translate-x-1/2 w-[600px] h
-[22px] rounded-[50%] bg-[#4A3A2A]/35 blur-[6px]" />

          {/* 右侧方形分享按钮 */}
          <button
            type="button"
            onClick={handleShare}
            className="absolute -right-[136px] top-1/2 -translate-y-1/2 rotate-[-12deg]
z-10 flex flex-col items-center justify-center w-[112px] h-[86px] rounded-[16px] bg-gra
dient-to-b from-[#8B5CF6] to-[#6D28D9] text-white shadow-[0_3px_8px_rgba(0,0,0,0.3)] ho
ver:brightness-110 active:scale-95 transition-all"
          >
            <UiIcon name="share" size={20} />
            <span className="text-[15px] font-bold leading-[1.2] text-center mt-1">
              分享我的<br />徽章墙
            </span>
          </button>
        </div>
      </div>
    </div>
  )
}
import { useState } from 'react'
import { UiIcon } from '@/lib/uiIcons'
import type { TaskId, TaskStatus } from '@/types/checkin'
```

```
export interface GardenSubItem {
  key: string
  label: string
  done: boolean
}

interface Props {
  id: TaskId
  label: string
  img?: string
  gradient: 'water' | 'diet' | 'sleep' | 'exercise' | 'garden'
  accent: string
  xp: number
  status: TaskStatus
  tip: string
  gardenSubs?: GardenSubItem[]
  allGardenDone?: boolean
  editing: boolean
  onToggle: () => void
  onToggleSub?: (key: string) => void
}

export default function CareTaskCard({
  id,
  label,
  img,
  gradient,
  xp,
  status,
  gardenSubs,
  allGardenDone,
  editing,
  onToggle,
  onToggleSub,
}: Props) {
  const isDone = status === 'done' || status === 'makeup'
  const pending = status === 'pending'
  const pillText = isDone ? (status === 'makeup' ? '已补卡' : '已完成') : '去完成'
  const [subsOpen, setSubsOpen] = useState(false)

  const handlePrimary = () => {
    if (editing) return
    if (isDone) {
      onToggle()
      return
    }
    // 探索花园：未完成全部小任务前，点√只是展开小任务
    if (id === 'task_garden' && gardenSubs && !allGardenDone) {
      setSubsOpen((v) => !v)
      return
```

```
    }
    onToggle()
  }

  return (
    <article
      className={`relative h-full overflow-hidden rounded-xl shadow-[0_4px_14px_rgba(0,
0,0,0.16)] ${
        !editing && pending ? 'cursor-pointer' : ''
      }`}
      onClick={() => {
        if (editing) return
        if (pending) onToggle()
      }}
    >
      {/* 卡面 = 整张素材铺满卡片，完整显示 */}
      {img ? (
        <img src={img} alt={label} className="absolute inset-x-0 top-0 w-full h-[112%]
object-cover" draggable={false} />
      ) : (
        <div className={`absolute inset-0 w-full h-full ggc-task-${gradient}`} />
      )}

      {/* 花园小任务 chips：点√后展开，叠加在卡片图片下部 */}
      {id === 'task_garden' && gardenSubs && subsOpen && !isDone && (
        <div className="absolute inset-x-0 bottom-[74px] flex justify-center gap-1 px-2
z-10">
          {gardenSubs.map((s) => (
            <button
              key={s.key}
              className={`inline-flex items-center justify-center gap-1 rounded-full bo
rder px-1.5 py-0.5 text-[10px] font-medium shadow-md transition-colors ${
                s.done
                  ? 'bg-[#d9efbe] border-green-300 text-green-700'
                  : 'bg-white border-gray-200 text-gray-600'
              }`}
              onClick={(e) => {
                if (editing) return
                e.stopPropagation()
                onToggleSub?.(s.key)
              }}
            >
              <span>{s.done ? '✓' : 'o'}</span>
              <span className="truncate">{s.label}</span>
            </button>
          ))}
        </div>
      )}

      {/* 底部控制区：叠加在卡片图片内部，居中 */}
      <div className="absolute inset-x-0 bottom-0 flex items-center justify-center pb-2
z-10">
        <div className="flex items-center gap-1.5">
          <button
            className="w-8 h-8 rounded-full flex items-center justify-center shadow-md
transition-transform hover:scale-105 active:scale-95 shrink-0"
```

```
            style={
              isDone
                ? { background: '#4caf50', color: '#fff', border: 'none' }
                : { background: '#fff', color: '#4caf50', border: '2px solid #4caf50' }
            }
            onClick={(e) => {
              e.stopPropagation()
              handlePrimary()
            }}
            title={isDone ? '取消完成' : id === 'task_garden' && gardenSubs && !allGarden
Done ? '查看小任务' : '完成'}
          >
            <UiIcon name="check" size={16} strokeWidth={3.2} />
          </button>

          <span
            className={`w-20 text-[11px] font-bold px-2 py-1 rounded-full border text-c
enter whitespace-nowrap shadow-md ${
              isDone ? 'bg-[#4caf50] border-[#4caf50] text-white' : 'bg-white border-gr
ay-200 text-gray-600'
            }`}
          >
            {pillText}
          </span>

          <span className="inline-flex items-center gap-0.5 text-[11px] font-bold text-
green-700 bg-white border border-gray-100 rounded-full px-2 py-1 shrink-0 whitespace-no
wrap shadow-md">
            <UiIcon name="zap" size={11} className="text-amber-500" />
            +{xp}
          </span>
        </div>
      </div>
    </article>
  )
}
import { UiIcon } from '@/lib/uiIcons'

const ITEMS = [
  { icon: 'droplet', label: '水滴能量', value: 5, color: '#38bdf8' },
  { icon: 'sprout', label: '植物成长', value: 10, color: '#4caf50' },
  { icon: 'starGold', label: '菌群快乐', value: 8, color: '#f5bd35' },
  { icon: 'sun', label: '花园币', value: 5, color: '#fb923c' },
]

export default function DailyReward() {
  const total = ITEMS.reduce((s, i) => s + i.value, 0)
  return (
    <div className="ggc-card bg-[rgba(255,249,226,0.9)] backdrop-blur-md flex flex-col
h-full p-4 overflow-hidden">
      <h3 className="text-[16px] font-bold text-gray-700">今日奖励</h3>
      <p className="text-[11px] text-gray-400 mt-0.5">今日获得</p>
      <div className="flex-1 flex flex-col justify-center gap-2 mt-1">
        {ITEMS.map((it) => (
          <div key={it.label} className="flex items-center gap-2.5 bg-white/60 rounded-
2xl px-3 py-2 shadow-sm">
            <UiIcon name={it.icon} size={22} />
```

```
            <span className="text-[13px] text-gray-600 flex-1">{it.label}</span>
            <span className="text-[13px] font-bold" style={{ color: it.color }}>+{it.va
lue}</span>
          </div>
        ))}
      </div>
      <div className="mt-2 pt-2 border-t border-white/70 flex items-center justify-betw
een px-1">
        <span className="text-[13px] text-gray-500">总计</span>
        <span className="text-[24px] font-extrabold text-green-600 leading-none">+{tota
l}</span>
      </div>
    </div>
  )
}
interface Props {
  energy: number
}

export default function EnergyCard({ energy }: Props) {
  const pct = Math.round(Math.min(100, Math.max(0, energy)))
  return (
    <div className="flex flex-col items-center justify-center px-6 h-full relative over
flow-hidden">
      <img src="/assets/tasks/task_bg2.png" alt="" className="absolute inset-0 w-full h
-full object-fill" draggable={false} />
      <h3 className="relative z-10 text-[15px] font-bold text-gray-600 tracking-wide">今
日花园能量</h3>
      <div className="relative z-10 flex items-baseline leading-none mt-1">
        <span className="ggc-energy-num">{pct}</span>
        <span className="ggc-energy-total text-[18px] font-bold ml-1">/ 100</span>
      </div>
      <div className="relative z-10 ggc-energy-progress w-full mt-3">
        <div style={{ width: `${pct}%` }} />
      </div>
      <p className="relative z-10 text-[14px] font-semibold text-green-800 mt-2.5">距离
花园开花还差 {100 - pct} 能量 🌸</p>
    </div>
  )
}
import { UiIcon } from '@/lib/uiIcons'
import { useUIStore } from '@/stores/uiStore'

type StoolHealth = 'normal' | 'constipation' | 'diarrhea' | 'none'

interface Props {
  careDone: number
  total: number
  stoolHealth: StoolHealth
  onClassroom: () => void
  onBadges: () => void
}

export default function GardenAssistant({ careDone, total, stoolHealth, onClassroom, on
Badges }: Props) {
  const soundEnabled = useUIStore((s) => s.soundEnabled)
  const setSoundEnabled = useUIStore((s) => s.setSoundEnabled)
  const reportMode = stoolHealth === 'constipation' || stoolHealth === 'diarrhea'
```

```
  const message = reportMode
    ? '昨天的便便显示需要调整一下，今天重点关注饮食和水分哦！'
    : `早上好，小主人！🌼 今天已完成 ${careDone}/${total} 项任务，花园变得更漂亮啦！`

  const tips =
    stoolHealth === 'constipation'
      ? ['多吃绿叶蔬菜和纤维食物', '今天喝够 8 杯水', '饭后散步 15 分钟']
      : stoolHealth === 'diarrhea'
        ? ['清淡饮食，避免生冷', '注意腹部保暖', '适量补充温水']
        : ['先完成花园探索再吃饭', '记得喝够 6 杯水']

  return (
    <div className="ggc-assistant flex flex-col h-full p-5 overflow-hidden">
      {/* 头部 */}
      <div className="flex items-center justify-between shrink-0">
        <div className="flex items-center gap-2.5">
          <span className="w-12 h-12 rounded-full bg-white/70 flex items-center justify
-center shadow-md overflow-hidden shrink-0">
            <img src="/assets/characters/png/char_xiaoyuan.webp" alt="菌小园助手" classNa
me="w-11 h-11 object-contain" draggable={false} />
          </span>
          <div>
            <h3 className="text-[17px] font-bold text-green-800">菌小园助手</h3>
            <p className="text-[11px] text-gray-500">你的打卡小帮手</p>
          </div>
        </div>
        <button
          className="w-9 h-9 rounded-full bg-white/60 flex items-center justify-center
text-gray-400 shadow-sm shrink-0 active:scale-90 transition-transform"
          onClick={() => setSoundEnabled(!soundEnabled)}
          title={soundEnabled ? '关闭音效' : '开启音效'}
        >
          <UiIcon name={soundEnabled ? 'volumeLine' : 'volumeMuteLine'} size={18} />
        </button>
      </div>

      {/* 中段内容 */}
      <div className="flex-1 min-h-0 flex flex-col justify-center gap-3">
        <div className="bg-white/80 rounded-2xl rounded-bl-sm p-3 shadow-sm">
          <p className="text-[13px] text-gray-600 leading-relaxed">{message}</p>
        </div>

        <div>
          <p className="text-[13px] font-bold text-amber-700 inline-flex items-center g
ap-1">
            <UiIcon name="lightbulb" size={14} className="text-amber-500" />
            今日小建议
          </p>
          <ul className="text-[12px] text-gray-500 mt-1.5 space-y-1">
            {tips.map((t) => (
              <li key={t}>• {t}</li>
            ))}
          </ul>
```

```
        </div>

        <div>
          <p className="text-[13px] font-bold text-green-700 inline-flex items-center g
ap-1">
            <UiIcon name="book" size={14} className="text-green-600" />
            今日小知识
          </p>
          <p className="text-[12px] text-gray-500 mt-1.5 leading-relaxed">
            膳食纤维是益生菌最喜欢的食物，可以帮助肠道更健康!
          </p>
        </div>

        <button
          className="w-full py-2.5 bg-[#4CAF50] text-white rounded-2xl text-[14px] font
-bold shadow-md hover:bg-[#43A047] active:scale-95 transition-all"
          onClick={onClassroom}
        >
          去知识课堂看看 →
        </button>
      </div>

      {/* 快捷入口 */}
      <div className="mt-3 pt-3 border-t border-white/60 shrink-0">
        <p className="text-[12px] text-gray-500 mb-2">快捷入口</p>
        <div className="grid grid-cols-2 gap-2">
          <button
            className="flex items-center gap-2 bg-white/70 rounded-2xl px-3 py-2 hover:
bg-white active:scale-95 transition-all"
            onClick={onClassroom}
          >
            <UiIcon name="book" size={18} className="text-green-600" />
            <span className="text-[12px] text-gray-600">知识课堂</span>
          </button>
          <button
            className="flex items-center gap-2 bg-white/70 rounded-2xl px-3 py-2 hover:
bg-white active:scale-95 transition-all"
            onClick={onBadges}
          >
            <UiIcon name="trophy" size={18} className="text-amber-500" />
            <span className="text-[12px] text-gray-600">成长徽章</span>
          </button>
        </div>
      </div>
    </div>
  )
}
import { UiIcon } from '@/lib/uiIcons'

interface Props {
  level: number
  stageName: string
  xpToNext: number
  xpPct: number
```

```
  plantGrowth: number
  vitality: number
  weekLabel: string
}

export default function GardenGrowthCard({ level, stageName, xpToNext, xpPct, plantGrow
th, vitality, weekLabel }: Props) {
  const stats = [
    { icon: '🌳', label: '植物成长', value: `${plantGrowth}%`, color: 'text-[#5b9d3b]' },
    { icon: '🌸', label: '花园活力', value: `${vitality}%`, color: 'text-[#4c9a2f]' },
    { icon: '⭐', label: '本周表现', value: weekLabel, color: 'text-[#6b9c3b]' },
  ]
  const pct = Math.round(Math.min(100, Math.max(0, xpPct)))
  return (
    <div className="garden-growth-card relative flex flex-col h-full overflow-hidden">
      {/* 顶部标题 */}
      <h2 className="growth-title relative z-10 flex items-center gap-1.5 pt-[14px] px-
[16px] shrink-0">
        我的花园成长
        <UiIcon name="sprout" size={18} className="text-[#6a9f3d]" />
      </h2>

      {/* 主体：左侧成长状态 + 右侧指标 */}
      <div className="relative z-10 flex-1 flex min-h-0 px-[14px] pt-[4px]">
        {/* 左侧：植物插画 + 等级 + 阶段 */}
        <div className="w-[48%] flex flex-col items-center justify-center">
          <span className="growth-plant leading-none drop-shadow-[0_5px_8px_rgba(90,13
0,50,0.35)]">🌳</span>
          <div className="flex items-baseline gap-[5px] mt-[6px]">
            <span className="level-label">生态等级</span>
            <strong className="level-num">Lv.{level}</strong>
          </div>
          <span className="stage-label mt-[5px]">{stageName}</span>
        </div>

        {/* 右侧：三个成长指标 */}
        <div className="flex-1 flex flex-col justify-center gap-[8px] pl-[10px] min-w-
0">
          {stats.map((s) => (
            <div key={s.label} className="stat-item">
              <span className="stat-icon">{s.icon}</span>
              <span className="stat-name">{s.label}</span>
              <strong className={`stat-value ${s.color}`}>{s.value}</strong>
            </div>
          ))}
        </div>
      </div>

      {/* 底部 XP 进度 */}
      <div className="relative z-10 px-[16px] pb-[10px] pt-[4px] shrink-0">
        <div className="xp-progress">
          <div className="xp-progress-fill" style={{ width: `${pct}%` }} />
        </div>
        <p className="xp-text">
```

```
          距离 Lv.{level + 1} 还有 <strong>{xpToNext} XP</strong>
        </p>
      </div>

      {/* 装饰 */}
      <UiIcon name="leaf" size={22} className="garden-growth-deco top-[10px] right-[12p
x] opacity-[0.55]" />
      <UiIcon name="flower" size={20} className="garden-growth-deco bottom-[34px] right
-[10px] opacity-[0.4]" />
    </div>
  )
}
interface Props {
  level: number
}

export default function GardenPreviewCard({ level }: Props) {
  return (
    <div className="ggc-card bg-[rgba(255,250,235,0.9)] backdrop-blur-md flex flex-col
items-center justify-center h-full p-2 relative overflow-hidden">
      <h3 className="text-[14px] font-bold text-gray-500 mb-0.5">花园预览</h3>
      <img
        src="/assets/ui/ui_kingkong_garden.webp"
        alt="花园预览"
        className="w-[82%] object-contain drop-shadow-md"
        draggable={false}
      />
      <span className="absolute top-1.5 right-1.5 text-[9px] font-bold text-white bg-[#
4CAF50]/90 rounded-full px-1.5 py-0.5 shadow-sm">
        Lv.{level}
      </span>
    </div>
  )
}
interface Props {
  level: number
  stageName: string
}

export default function LevelCard({ level, stageName }: Props) {
  return (
    <div className="ggc-level-card ggc-clay flex items-center gap-3 px-5 h-full overflo
w-hidden">
      <span className="text-[42px] leading-none drop-shadow-sm shrink-0">🌱</span>
      <div className="min-w-0">
        <p className="text-[13px] font-semibold text-gray-500">生态等级</p>
        <p className="text-[30px] font-extrabold leading-tight text-green-700">Lv.{leve
l}</p>
        <p className="text-[13px] text-green-600 mt-0.5 whitespace-nowrap">{stageName}
</p>
      </div>
    </div>
  )
}
interface Props {
  streak: number
}
```

```
const WEEKDAY = ['一', '二', '三', '四', '五', '六', '日']

export default function StreakCalendar({ streak }: Props) {
  const today = new Date()
  today.setHours(0, 0, 0, 0)
  const year = today.getFullYear()
  const month = today.getMonth()
  const daysInMonth = new Date(year, month + 1, 0).getDate()
  const firstWeekday = (new Date(year, month, 1).getDay() + 6) % 7 // 周一=0

  const cells: (number | null)[] = [
    ...Array.from({ length: firstWeekday }, () => null),
    ...Array.from({ length: daysInMonth }, (_, i) => i + 1),
  ]
  while (cells.length % 7 !== 0) cells.push(null)
  const rows = cells.length / 7

  return (
    <div className="ggc-card bg-[rgba(255,249,226,0.9)] backdrop-blur-md flex flex-col
h-full p-3 overflow-hidden">
      <div className="flex items-center justify-between shrink-0">
        <h3 className="text-[15px] font-bold text-gray-700 inline-flex items-center gap
-1">🗓️ {year}年{month + 1}月</h3>
        <p className="text-[13px] font-bold text-orange-600">连续 {streak} 天 🔥</p>
      </div>

      <div className="grid grid-cols-7 text-center shrink-0 mt-1.5">
        {WEEKDAY.map((w) => (
          <span key={w} className="text-[11px] font-medium text-gray-400">{w}</span>
        ))}
      </div>

      <div className="flex-1 grid grid-cols-7 gap-0.5 mt-1 text-center" style={{ gridTe
mplateRows: `repeat(${rows}, 1fr)` }}>
        {cells.map((d, i) => {
          if (d === null) return <div key={i} />
          const date = new Date(year, month, d)
          const diff = Math.round((today.getTime() - date.getTime()) / 86400000)
          const isToday = diff === 0
          const isChecked = diff >= 0 && diff < streak
          const isFuture = diff < 0
          let cls = 'text-gray-500'
          if (isToday) {
            cls = 'bg-gradient-to-b from-orange-400 to-orange-500 text-white shadow'
          } else if (isChecked) {
            cls = 'bg-[#d9efbe] text-green-700'
          } else if (isFuture) {
            cls = 'text-gray-300'
          }
          return (
            <div key={i} className={`flex items-center justify-center rounded-[8px] ${c
ls}`}>
              <span className="text-[13px] font-bold leading-none">{d}</span>
```

```
            </div>
          )
        })}
      </div>

      <p className="mt-auto pt-1.5 text-[10px] text-amber-600 inline-flex items-center
gap-1">🎁 连续 7 天可获得神秘种子哦！</p>
    </div>
  )
}
import { useNavigate } from 'react-router-dom'
import { UiIcon } from '@/lib/uiIcons'
import TopRightControls from '@/components/navigation/TopRightControls'

export default function TopHeader() {
  const navigate = useNavigate()
  return (
    <header className="shrink-0 flex items-center justify-between px-6 py-2">
      <div className="flex items-center gap-2">
        <button
          className="w-9 h-9 rounded-full bg-garden-mascot text-white shadow-md hover:b
g-[#7A9538] active:scale-95 transition-all flex items-center justify-center"
          onClick={() => navigate('/')}
          title="返回首页"
        >
          <UiIcon name="chevronLeft" size={20} />
        </button>
        <div className="leading-tight">
          <div className="flex items-center gap-1.5">
            <p className="font-bold text-lg text-garden-forest">每日打卡</p>
            <span className="w-4 h-4 rounded-full bg-sky-500 text-white text-[10px] fle
x items-center justify-center leading-none">?</span>
          </div>
          <p className="text-[10px] text-gray-500">每天照顾一点点，花园会更好哦！</p>
        </div>
      </div>
      <TopRightControls />
    </header>
  )
}
import { useGardenStore } from '@/stores/gardenStore'

const STATE_SPRITES: Record<string, string> = {
  healthy: '/assets/characters/lottie/char_xiaoyuan_idle.webp',
  high_sugar: '/assets/characters/lottie/char_xiaoyuan_worry.webp',
  dry: '/assets/characters/lottie/char_xiaoyuan_worry.webp',
  recovering: '/assets/characters/png/char_xiaoyuan.webp',
}

export function Character() {
  const currentState = useGardenStore((s) => s.currentState)
  const sprite = STATE_SPRITES[currentState] || STATE_SPRITES.healthy
```

```
  return (
    <div className="absolute bottom-24 left-1/2 -translate-x-1/2 z-20 pointer-events-no
ne">
      <div className="relative">
        <div className="w-24 h-24 flex items-center justify-center">
          <img
            src={sprite}
            alt="小圆"
            className="w-full h-full object-contain animate-float"
          />
        </div>
      </div>
    </div>
  )
}
import { useDroppable } from '@dnd-kit/core'
import { UiIcon } from '@/lib/uiIcons'

export function DropZone() {
  const { setNodeRef, isOver } = useDroppable({ id: 'garden-drop-zone' })

  return (
    <div
      ref={setNodeRef}
      className={`absolute inset-0 z-10 transition-colors duration-300 rounded-2xl ${
        isOver ? 'bg-garden-mascot/10 ring-2 ring-garden-forest/40 ring-dashed' : ''
      }`}
    >
      {isOver && (
        <div className="absolute inset-0 flex items-center justify-center pointer-event
s-none">
          <span className="animate-bounce text-gray-400"><UiIcon name="utensils" size=
{36} strokeWidth={1.4} /></span>
        </div>
      )}
    </div>
  )
}
import { useDraggable } from '@dnd-kit/core'

const FOODS = [
  { name: 'broccoli', label: '西兰花', effect: 'healthy' },
  { name: 'carrot', label: '胡萝卜', effect: 'healthy' },
  { name: 'yogurt', label: '酸奶', effect: 'healthy' },
  { name: 'apple', label: '苹果', effect: 'healthy' },
  { name: 'corn', label: '玉米', effect: 'healthy' },
  { name: 'candy', label: '糖果', effect: 'high_sugar' },
  { name: 'cake', label: '蛋糕', effect: 'high_sugar' },
]

function DraggableFood({ name, label }: { name: string; label: string }) {
  const { attributes, listeners, setNodeRef, transform } = useDraggable({ id: name })
  const style = transform ? { transform: `translate(${transform.x}px, ${transform.y}px)
` } : undefined
```

```
  return (
    <button
      ref={setNodeRef}
      {...listeners}
      {...attributes}
      style={style}
      className="w-12 h-12 rounded-full bg-white/80 shadow flex items-center justify-ce
nter hover:scale-110 transition-transform cursor-grab active:cursor-grabbing"
    >
      <img src={`/assets/foods/food_${name}.png`} alt={label} className="w-8 h-8" />
    </button>
  )
}

export default function FoodToolbar() {
  return (
    <div className="flex items-center gap-2">
      {FOODS.map((f) => (
        <DraggableFood key={f.name} name={f.name} label={f.label} />
      ))}
    </div>
  )
}
import { useRef } from 'react'
import { useParallax } from '@/hooks/useParallax'
import { useGardenScene } from '@/hooks/useGardenScene'

export default function GardenStage() {
  const containerRef = useRef<HTMLDivElement>(null)
  const { skySrc, midSrc, frontSrc } = useGardenScene()

  const LAYERS = [
    { name: 'garden_sky', src: skySrc, translateZ: -200, speed: 0.1 },
    { name: 'garden_mid', src: midSrc, translateZ: 0, speed: 0.4 },
    { name: 'garden_front', src: frontSrc, translateZ: 150, speed: 1.0 },
  ]

  useParallax(containerRef)

  return (
    <div ref={containerRef} className="absolute inset-0 overflow-hidden"
      style={{ perspective: 1200, perspectiveOrigin: '50% 50%' }}>
      {LAYERS.map((l) => (
        <img
          key={l.name}
          src={l.src}
          data-parallax={l.speed}
          className="absolute inset-0 w-full h-full object-cover transition-all duratio
n-1000"
          style={{ transform: `translateZ(${l.translateZ}px) scale(${1 + l.translateZ /
800})` }}
        />
```

```
      ))}
    </div>
  )
}
import { useEffect, useRef } from 'react'
import { UiIcon } from '@/lib/uiIcons'

interface LottiePlayerProps {
  src?: string
  autoplay?: boolean
  loop?: boolean
  className?: string
}

export function LottiePlayer({ src, autoplay = true, loop = true, className = '' }: Lot
tiePlayerProps) {
  const containerRef = useRef<HTMLDivElement>(null)

  useEffect(() => {
    if (!src || !containerRef.current) return
    // Placeholder for dotLottie-web or lottie-web integration
    // Will load and play Lottie JSON from src when animated assets are ready
  }, [src])

  if (!src) {
    return (
      <div ref={containerRef} className={`flex items-center justify-center ${className}
`}>
        <span className="animate-pulse text-gray-400"><UiIcon name="clapperboard" size=
{36} strokeWidth={1.4} /></span>
      </div>
    )
  }

  return <div ref={containerRef} className={className} />
}
import { useEffect, useRef } from 'react'

interface Particle {
  x: number
  y: number
  vx: number
  vy: number
  size: number
  opacity: number
  life: number
  maxLife: number
}

export function ParticleLayer() {
  const canvasRef = useRef<HTMLCanvasElement>(null)

  useEffect(() => {
```

```
const canvas = canvasRef.current
if (!canvas) return

const ctx = canvas.getContext('2d')
if (!ctx) return

const resize = () => {
  canvas.width = canvas.parentElement!.clientWidth
  canvas.height = canvas.parentElement!.clientHeight
}
resize()
window.addEventListener('resize', resize)

const particles: Particle[] = []
const MAX_PARTICLES = 30

const spawn = () => {
  if (particles.length >= MAX_PARTICLES) return
  particles.push({
    x: Math.random() * canvas.width,
    y: canvas.height + 10,
    vx: (Math.random() - 0.5) * 0.4,
    vy: -(Math.random() * 0.6 + 0.2),
    size: Math.random() * 3 + 1,
    opacity: Math.random() * 0.4 + 0.1,
    life: 0,
    maxLife: Math.random() * 200 + 100,
  })
}

let animId: number
const animate = () => {
  ctx.clearRect(0, 0, canvas.width, canvas.height)

  // Occasionally spawn
  if (Math.random() < 0.3) spawn()

  for (let i = particles.length - 1; i >= 0; i--) {
    const p = particles[i]
    p.x += p.vx
    p.y += p.vy
    p.life++

    const alpha = p.opacity * (1 - p.life / p.maxLife)
    ctx.beginPath()
    ctx.arc(p.x, p.y, p.size, 0, Math.PI * 2)
    ctx.fillStyle = `rgba(184, 212, 227, ${alpha})`
    ctx.fill()

    if (p.life >= p.maxLife) particles.splice(i, 1)
```

```
      }

      animId = requestAnimationFrame(animate)
    }

    animId = requestAnimationFrame(animate)

    return () => {
      cancelAnimationFrame(animId)
      window.removeEventListener('resize', resize)
    }
  }, [])

  return <canvas ref={canvasRef} className="absolute inset-0 z-10 pointer-events-none"
/>
}
import { UiIcon } from '@/lib/uiIcons'

interface POITagProps {
  name: string
  x: number
  y: number
  unlocked: boolean
  onClick?: () => void
}

export function POITag({ name, x, y, unlocked, onClick }: POITagProps) {
  return (
    <button
      className={`absolute z-20 transition-all duration-300 ${
        unlocked
          ? 'opacity-100 hover:scale-110'
          : 'opacity-40 grayscale cursor-not-allowed'
      }`}
      style={{ left: `${x}%`, top: `${y}%`, transform: 'translate(-50%, -50%)' }}
      onClick={unlocked ? onClick : undefined}
      disabled={!unlocked}
    >
      <div className={`px-3 py-1.5 rounded-full text-xs font-bold whitespace-nowrap sha
dow-md ${
        unlocked
          ? 'bg-white/90 text-garden-forest'
          : 'bg-gray-200 text-gray-400'
      }`}>
        {unlocked ? name : <span className="inline-flex items-center gap-1"><UiIcon nam
e="lock" size={14} /> 未解锁</span>}
      </div>
    </button>
  )
}
import { lazy, Suspense, useEffect } from 'react'
import { Outlet, useLocation } from 'react-router-dom'
import { ReadingLevelProvider } from '@/providers/ReadingLevelProvider'
```

```
import { ToastContainer } from '@/components/ui/Toast'
import { useUIStore } from '@/stores/uiStore'
import { useApiSync } from '@/hooks/useApiSync'
import BottomDock from '@/components/navigation/BottomDock'

const StoolModal = lazy(() => import('@/components/modals/StoolModal'))
const AIChatModal = lazy(() => import('@/components/modals/AIChatModal'))
const OnboardingOverlay = lazy(() => import('@/components/onboarding/OnboardingOverla
y'))

export default function Layout() {
  const location = useLocation()
  const sceneWallpapers: Record<string, string> = {
    '/': '/assets/scenes/scene_home_bg.webp',
    '/checkin': '/assets/scenes/scene_checkin_bg.webp',
    '/profile': '/assets/scenes/scene_checkin_bg.webp',
    '/badges': '/assets/scenes/scene_badge_bg.webp',
    '/garden': '/assets/scenes/scene_garden_bg.webp',
    '/classroom': '/assets/scenes/scene_classroom_map.webp',
  }
  const wallpaper = sceneWallpapers[location.pathname]
  const stoolModalOpen = useUIStore((s) => s.stoolModalOpen)
  const aiChatOpen = useUIStore((s) => s.aiChatOpen)
  const onboardingComplete = useUIStore((s) => s.onboardingComplete)
  const editing = useUIStore((s) => s.editing)
  const toggleEditing = useUIStore((s) => s.toggleEditing)
  useApiSync()

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      if (e.ctrlKey && e.key === 'e') {
        e.preventDefault()
        toggleEditing()
      }
    }
    window.addEventListener('keydown', onKey)
    return () => window.removeEventListener('keydown', onKey)
  }, [toggleEditing])

  return (
    <ReadingLevelProvider>
      <div className="h-screen flex items-center justify-center bg-garden-mascot p-4">
        <div
          className={`relative overflow-hidden rounded-[2rem] shadow-2xl w-[min(calc(10
0vw-2rem),calc((100vh-2rem)*16/10))] h-[min(calc(100vh-2rem),calc((100vw-2rem)*10/16))]
${wallpaper ? '' : 'bg-garden-cream'}`}
          style={wallpaper ? {
            backgroundImage: `url(${wallpaper})`,
            backgroundSize: 'cover',
            backgroundPosition: 'center bottom',
            backgroundRepeat: 'no-repeat',
          } : undefined}
        >
```

```
          <div className="w-full h-full flex flex-col">
            {/* Global edit mode banner */}
            {editing && (
              <div className="bg-garden-coral/90 text-white text-[11px] text-center py-
0.5 font-medium flex items-center justify-center gap-3 shrink-0">
                <span>Edit Mode — Ctrl+E to exit</span>
              </div>
            )}
            <div className="flex-1 overflow-auto">
              <Outlet />
            </div>
            <BottomDock />
          </div>

          <ToastContainer />

          {stoolModalOpen && (
            <Suspense fallback={null}>
              <StoolModal />
            </Suspense>
          )}

          {aiChatOpen && (
            <Suspense fallback={null}>
              <AIChatModal />
            </Suspense>
          )}

          {!onboardingComplete && (
            <Suspense fallback={null}>
              <OnboardingOverlay />
            </Suspense>
          )}
        </div>
      </div>
    </ReadingLevelProvider>
  )
}
import { useEffect, useRef, useState } from 'react'
import { useLocation } from 'react-router-dom'
import { useUIStore } from '@/stores/uiStore'
import { apiStream } from '@/lib/api'
import { toast } from '@/components/ui/Toast'
import { UiIcon } from '@/lib/uiIcons'

interface Message {
  role: 'user' | 'ai'
  content: string
}

function pageFromPath(pathname: string): string {
```

```
  const map: Record<string, string> = {
    '/checkin': 'checkin',
    '/garden': 'garden',
    '/classroom': 'classroom',
    '/stool': 'stool',
    '/badge': 'badge',
    '/profile': 'profile',
    '/report': 'report',
  }
  return map[pathname] ?? 'home'
}

const QUICK_QUESTIONS = ['为什么要吃蔬菜？', '酸奶有好处吗？', '为什么放屁会臭？', '每天要喝多少水？']

export default function AIChatModal() {
  const setAiChatOpen = useUIStore((s) => s.setAiChatOpen)
  const { pathname } = useLocation()
  const [messages, setMessages] = useState<Message[]>([
    { role: 'ai', content: '嗨！我是菌小园，你的肠道小导游～ 有什么想问的吗？' },
  ])
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)
  const bottomRef = useRef<HTMLDivElement>(null)

  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' })
  }, [messages, loading])

  const send = async (text: string) => {
    const content = text.trim()
    if (!content || loading) return
    setInput('')
    setMessages((prev) => [...prev, { role: 'user', content }])
    setLoading(true)
    setMessages((prev) => [...prev, { role: 'ai', content: '' }])

    const history = messages
      .filter((m) => m.content.trim())
      .slice(-6)
      .map((m) => ({ role: m.role === 'ai' ? 'assistant' : 'user', content: m.content
}))

    let acc = ''
    await apiStream(
      '/ai/chat',
      { message: content, page: pageFromPath(pathname), history },
      (chunk) => {
        acc += chunk
        setMessages((prev) => {
          const next = [...prev]
          next[next.length - 1] = { role: 'ai', content: acc }
```

```
          return next
        })
      },
      () => {
        if (!acc) toast('菌小园开小差了，稍后再试试吧～', 'error')
      }
    )
    setLoading(false)
  }

  return (
    <div className="absolute inset-0 z-50 flex items-center justify-center">
      <div className="absolute inset-0 bg-black/50 backdrop-blur-sm" onClick={() => set
AiChatOpen(false)} />
      <div className="relative bg-white/95 backdrop-blur rounded-2xl shadow-2xl w-full
max-w-[420px] max-h-[80vh] mx-4 flex flex-col overflow-hidden">
        <div className="flex items-center justify-between p-4 border-b border-gray-100
shrink-0">
          <div className="flex items-center gap-2">
            <img src="/assets/characters/png/char_xiaoyuan.webp" alt="菌小园" className
="w-10 h-10 object-contain" />
            <div>
              <h2 className="font-bold text-garden-forest text-sm">菌小园</h2>
              <p className="text-[10px] text-gray-400">你的肠道小导游</p>
            </div>
          </div>
          <button className="text-gray-400 hover:text-gray-600" onClick={() => setAiCha
tOpen(false)}><UiIcon name="close" size={20} /></button>
        </div>

        <div className="flex-1 overflow-auto p-4 space-y-3 min-h-[240px]">
          {messages.map((m, i) => (
            <div key={i} className={`flex ${m.role === 'user' ? 'justify-end' : 'justif
y-start'}`}>
              <div
                className={`max-w-[80%] rounded-2xl px-3.5 py-2.5 text-xs leading-relax
ed whitespace-pre-wrap ${
                  m.role === 'user'
                    ? 'bg-garden-mascot text-white rounded-br-sm'
                    : 'bg-garden-cream text-gray-700 rounded-bl-sm'
                }`}
              >
                {m.content || (loading && i === messages.length - 1 ? '...' : '')}
              </div>
            </div>
          ))}
          <div ref={bottomRef} />
        </div>

        {messages.length <= 1 && (
          <div className="px-4 pb-3 flex flex-wrap gap-1.5 shrink-0">
            {QUICK_QUESTIONS.map((q) => (
              <button
                key={q}
                className="text-[11px] text-garden-forest bg-garden-cream rounded-full
px-3 py-1.5 hover:bg-garden-sky/40 transition-colors"
                onClick={() => send(q)}
              >
```

```
                {q}
              </button>
            ))}
          </div>
        )}

        <div className="p-4 border-t border-gray-100 shrink-0 flex gap-2">
          <input
            className="flex-1 border border-gray-200 rounded-xl px-3.5 py-2.5 text-sm f
ocus:outline-none focus:border-garden-forest transition-colors"
            placeholder="问问菌小园..."
            value={input}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && send(input)}
          />
          <button
            className="px-4 py-2.5 rounded-xl text-sm font-bold bg-garden-mascot text-w
hite hover:bg-[#7A9538] active:scale-95 transition-all disabled:opacity-50"
            onClick={() => send(input)}
            disabled={!input.trim() || loading}
          >
            发送
          </button>
        </div>
      </div>
    </div>
  )
}
import { useState, useCallback } from 'react'
import { useUIStore } from '@/stores/uiStore'
import { useAuthStore } from '@/stores/authStore'
import { toast } from '@/components/ui/Toast'
import { api } from '@/lib/api'
import { isRegistered, getActiveChildId } from '@/hooks/useApiSync'
import { UiIcon } from '@/lib/uiIcons'

type Mode = 'icon' | 'photo'

interface BristolType {
  id: number
  label: string
  icon: string
  desc: string
  health: 'good' | 'ok' | 'bad'
}

const STOOL_PNG: Record<number, string> = {
  1: '/assets/stools/stool_type1_rabbit.webp',
  2: '/assets/stools/stool_type2_grape.webp',
  3: '/assets/stools/stool_type3_corn.webp',
  4: '/assets/stools/stool_type4_banana.webp',
  5: '/assets/stools/stool_type5_icecream.webp',
```

```
  6: '/assets/stools/stool_type6_marshmallow.webp',
  7: '/assets/stools/stool_type7_water.webp',
}

const BRISTOL_TYPES: BristolType[] = [
  { id: 1, label: '坚果便', icon: STOOL_PNG[1], desc: '干硬、分散的颗粒', health: 'bad' },
  { id: 2, label: '香肠便', icon: STOOL_PNG[2], desc: '干硬、表面凹凸', health: 'bad' },
  { id: 3, label: '条状有裂痕', icon: STOOL_PNG[3], desc: '表面有裂痕', health: 'ok' },
  { id: 4, label: '香蕉便', icon: STOOL_PNG[4], desc: '光滑柔软像香蕉', health: 'good' },
  { id: 5, label: '软块便', icon: STOOL_PNG[5], desc: '边缘清晰的软块', health: 'ok' },
  { id: 6, label: '糊状', icon: STOOL_PNG[6], desc: '边缘参差不齐', health: 'bad' },
  { id: 7, label: '水状', icon: STOOL_PNG[7], desc: '完全液体', health: 'bad' },
]

const HEALTH_RESULTS: Record<string, string> = {
  good: '香蕉便！非常健康！继续保持均衡饮食～',
  ok: '基本正常～注意多喝水、多吃蔬菜哦～',
  bad: '便便状态需要关注！建议调整饮食，多吃纤维食物，多喝水。持续异常请就医。',
}

export default function StoolModal() {
  const setStoolModalOpen = useUIStore((s) => s.setStoolModalOpen)
  const mode = useAuthStore((s) => s.mode)
  const [activeMode, setActiveMode] = useState<Mode>('icon')
  const [selected, setSelected] = useState<number | null>(null)
  const [result, setResult] = useState<string | null>(null)

  const handleSelect = useCallback((typeId: number) => {
    setSelected(typeId)
    const bristol = BRISTOL_TYPES.find((b) => b.id === typeId)
    if (bristol) {
      setResult(HEALTH_RESULTS[bristol.health] || '')
    }
  }, [])

  const handleConfirm = () => {
    if (selected === null) return

    if (isRegistered()) {
      const childId = getActiveChildId()
      if (childId) {
        api
          .post<{ bristol_type: number; diagnosis: string }>('/stool/select-icon', { ch
ild_id: childId, bristol_type: selected })
          .then((data) => {
            const bristol = BRISTOL_TYPES.find((b) => b.id === data.bristol_type)
            toast(`已记录：${bristol?.label} · ${data.diagnosis ?? ''}`, 'success')
            setStoolModalOpen(false)
          })
          .catch(() => {})
      }
```

```
      return
    }

    const entry = {
      date: new Date().toISOString().slice(0, 10),
      typeId: selected,
      time: new Date().toLocaleTimeString('zh-CN', { hour: '2-digit', minute: '2-digit'
}),
    }
    try {
      const saved = localStorage.getItem('gg-stool-logs')
      const logs = saved ? JSON.parse(saved) : []
      localStorage.setItem('gg-stool-logs', JSON.stringify([entry, ...logs]))
    } catch { /* ignore */ }
    const bristol = BRISTOL_TYPES.find((b) => b.id === selected)
    toast(`已记录：${bristol?.label}`, 'success')
    setStoolModalOpen(false)
  }

  return (
    <div className="absolute inset-0 z-50 flex items-start justify-center pt-8">
      {/* Overlay */}
      <div
        className="absolute inset-0 bg-black/50 backdrop-blur-sm"
        onClick={() => setStoolModalOpen(false)}
      />

      {/* Container */}
      <div className="relative parchment-card w-full max-w-[600px] mx-4 overflow-auto m
ax-h-[90vh]">
        {/* Title */}
        <div className="flex items-center justify-between p-5 border-b border-gray-10
0">
          <h2 className="text-lg font-bold text-garden-forest inline-flex items-center
gap-2"><UiIcon name="camera" size={18} /> 今日便便观察</h2>
          <button
            className="text-gray-400 hover:text-gray-600"
            onClick={() => setStoolModalOpen(false)}
          >
            <UiIcon name="close" size={20} />
          </button>
        </div>

        {/* Mode switch */}
        <div className="flex gap-1 bg-garden-cream rounded-xl p-1 m-5 mb-0">
          <button
            className={`flex-1 py-2.5 rounded-lg text-sm font-medium transition-colors
inline-flex items-center justify-center gap-1.5 ${
              activeMode === 'icon'
                ? 'bg-white text-garden-forest shadow-sm'
                : 'text-gray-600 hover:text-garden-forest/70'
            }`}
            onClick={() => setActiveMode('icon')}
          >
            <UiIcon name="rabbit" size={16} /> 图标选择
```

```
          </button>
          <button
            className={`flex-1 py-2.5 rounded-lg text-sm font-medium transition-colors
inline-flex items-center justify-center gap-1.5 ${
              activeMode === 'photo'
                ? 'bg-white text-garden-forest shadow-sm'
                : 'text-gray-600 hover:text-garden-forest/70'
            }`}
            onClick={() => setActiveMode('photo')}
            disabled={mode === 'guest'}
          >
            <UiIcon name="camera" size={16} /> 拍照分析
            {mode === 'guest' && <span className="text-[10px] ml-1">需注册</span>}
          </button>
        </div>

        {/* Content */}
        <div className="p-5">
          {activeMode === 'icon' ? (
            <div className="flex flex-wrap justify-center gap-4 mb-5 mx-auto" style={{
maxWidth: 480 }}>
              {BRISTOL_TYPES.map((b) => (
                <button
                  key={b.id}
                  className={`flex flex-col items-center gap-2 p-3 rounded-2xl border-2
transition-all w-[108px] ${
                    selected === b.id
                      ? 'border-garden-forest bg-garden-cream shadow-md scale-105'
                      : 'border-transparent bg-gray-50 hover:bg-white hover:border-gray
-200'
                  }`}
                  onClick={() => handleSelect(b.id)}
                >
                  <img src={b.icon} alt={b.label} className="w-[60px] h-[60px] object-c
ontain" />
                  <span className="text-[16px] font-medium text-gray-700 whitespace-now
rap">{b.label}</span>
                  <span className="text-[15px] text-gray-500">Type {b.id}</span>
                </button>
              ))}
            </div>
          ) : (
            <div className="border-2 border-dashed border-gray-200 rounded-xl p-8 text-
center mb-4">
              <span className="mb-3 block text-gray-300"><UiIcon name="camera" size={4
0} strokeWidth={1.4} /></span>
              <p className="text-sm text-gray-600 mb-1">
                {mode === 'guest'
                  ? '注册后可上传便便照片进行AI分析'
                  : '拖拽或点击上传 · ≤10MB自动压缩 · HEIC→JPEG'}
              </p>
              {mode === 'guest' ? (
                <p className="text-xs text-garden-coral">请先注册登录后使用拍照分析功能</p>
              ) : (
                <button className="mt-2 text-sm text-garden-forest hover:underline">选择
文件</button>
              )}
            </div>
          )}
```

```
import type { FastifyInstance } from 'fastify'
import { listChildren, createChild, updateChild, deleteChild } from './children.servic
e.js'
import { requireParentId } from '../auth/auth.routes.js'

export default async function childrenRoutes(fastify: FastifyInstance): Promise<void> {
  fastify.get('/api/children', async (req) => {
    const pid = requireParentId(req)
    const list = await listChildren(pid)
    return { code: 0, data: list }
  })

  fastify.post('/api/children', async (req) => {
    const pid = requireParentId(req)
    const body = req.body as { nickname?: string; age?: number; avatar_url?: string }
    if (!body.nickname || body.age === undefined) {
      return { code: 'VALIDATION', message: '昵称和年龄必填' }
    }
    const child = await createChild(pid, { nickname: body.nickname, age: body.age, avat
ar_url: body.avatar_url })
    return { code: 0, data: child }
  })

  fastify.put('/api/children/:id', async (req) => {
    const pid = requireParentId(req)
    const { id } = req.params as { id: string }
    const body = req.body as { nickname?: string; age?: number; avatar_url?: string | n
ull }
    const child = await updateChild(pid, Number(id), body)
    return { code: 0, data: child }
  })

  fastify.delete('/api/children/:id', async (req) => {
    const pid = requireParentId(req)
    const { id } = req.params as { id: string }
    await deleteChild(pid, Number(id))
    return { code: 0, data: { deleted: true } }
  })
}
import { eq, and } from 'drizzle-orm'
import { db } from '../../db/index.js'
import { children, gardenStates } from '../../db/schema/index.js'
import { toChildProfile, type ChildProfile } from '../../lib/mappers.js'
import { throwError } from '../../config/errors'

export interface CreateChildInput {
  nickname: string
  age: number
  avatar_url?: string | null
}

function assertAge(age: number): void {
  if (!Number.isInteger(age) || age < 3 || age > 10) throwError('CHILD_002')
```

```
}

export async function listChildren(parentId: number): Promise<ChildProfile[]> {
  const rows = await db.select().from(children).where(eq(children.parentId, parentId)).
orderBy(children.createdAt)
  return rows.map(toChildProfile)
}

export async function createChild(parentId: number, input: CreateChildInput): Promise<C
hildProfile> {
  assertAge(input.age)

  const [child] = await db.transaction(async (tx) => {
    const [created] = await tx
      .insert(children)
      .values({
        parentId,
        nickname: input.nickname,
        age: input.age,
        avatarUrl: input.avatar_url ?? null,
        dailyLimitMinutes: 30,
      })
      .returning()

    await tx
      .insert(gardenStates)
      .values({
        childId: created.id,
        currentState: 'healthy',
        moistureLevel: 50,
        growthStage: 1,
        gardenXp: 0,
        unlockedFeatures: [],
      })
      .onConflictDoNothing()

    return [created]
  })

  return toChildProfile(child)
}

export async function updateChild(parentId: number, childId: number, input: Partial<Cre
ateChildInput>): Promise<ChildProfile> {
  const existing = (await db.select().from(children).where(and(eq(children.id, childI
d), eq(children.parentId, parentId))))[0]
  if (!existing) throwError('CHILD_001')

  if (input.age !== undefined) assertAge(input.age)

  const [updated] = await db
    .update(children)
    .set({
      nickname: input.nickname ?? existing.nickname,
```

```
      age: input.age ?? existing.age,
      avatarUrl: input.avatar_url !== undefined ? (input.avatar_url ?? null) : existin
g.avatarUrl,
    })
    .where(eq(children.id, childId))
    .returning()

  return toChildProfile(updated)
}

export async function deleteChild(parentId: number, childId: number): Promise<void> {
  const existing = (await db.select().from(children).where(and(eq(children.id, childI
d), eq(children.parentId, parentId))))[0]
  if (!existing) throwError('CHILD_001')

  await db.delete(children).where(eq(children.id, childId))
}
export type ModuleCode = 'fiber_square' | 'ferment_workshop' | 'scfa_spring' | 'barrier
_wall' | 'eco_station'
export type QuizType = 'single_choice' | 'pairing' | 'ordering'

export interface CardContent {
  id: string
  title: string
  front_image: string
  back_content: string
  child_summary: string
  parent_detail: string
}

export interface QuizContent {
  id: string
  module_code: ModuleCode
  type: QuizType
  question: string
  options: string[]
  answer: number | number[]
  answer_hint: string
}

export interface ModuleDef {
  module_code: ModuleCode
  name: string
  description: string
  cards: CardContent[]
  quizzes: QuizContent[]
}

export const MODULE_ORDER: ModuleCode[] = ['fiber_square', 'ferment_workshop', 'scfa_sp
ring', 'barrier_wall', 'eco_station']

function card(module_code: ModuleCode, n: number, title: string, back_content: string,
child_summary: string, parent_detail: string): CardContent {
  return {
    id: `${module_code}_c${n}`,
```

```
    title,
    front_image: `/assets/cards/card_${module_code}.png`,
    back_content,
    child_summary,
    parent_detail,
  }
}

function quiz(
  module_code: ModuleCode,
  n: number,
  type: QuizType,
  question: string,
  options: string[],
  answer: number | number[],
  answer_hint: string
): QuizContent {
  return { id: `${module_code}_q${n}`, module_code, type, question, options, answer, an
swer_hint }
}

export const MODULE_DEFS: Record<ModuleCode, ModuleDef> = {
  fiber_square: {
    module_code: 'fiber_square',
    name: '膳食纤维广场',
    description: '了解膳食纤维如何喂养肠道居民',
    cards: [
      card(
        'fiber_square',
        1,
        '什么是膳食纤维？',
        '膳食纤维是植物里不容易被消化的部分，像一把小扫把，帮肠道做清洁，还能喂养肠道里的好居民。',
        '膳食纤维像小扫把，帮肚子做清洁～',
        '膳食纤维能促进肠道蠕动、增加饱腹感，是维持消化健康的重要成分。'
      ),
      card(
        'fiber_square',
        2,
        '纤维去哪里了？',
        '纤维吃进肚子后，一路到达大肠，在那里被益生菌们「吃掉」发酵，产生对身体有益的物质。',
        '纤维会坐着滑梯到达大肠，被小居民吃掉！',
        '纤维在小肠几乎不被吸收，主要在大肠被肠道菌群发酵利用。'
      ),
      card(
        'fiber_square',
        3,
        '蔬菜里的纤维',
        '西蓝花、胡萝卜、菠菜这些绿油油的蔬菜都藏着丰富的纤维，是肠道居民最爱的口粮之一。',
        '西蓝花和胡萝卜里藏着很多纤维口粮！',
        '深色蔬菜膳食纤维含量高，建议每天摄入 300-500g 蔬菜。'
      ),
```

```
      card(
        'fiber_square',
        4,
        '水果里的纤维',
        '苹果、香蕉、蓝莓等水果不仅好吃，果肉和果皮里也有很多纤维，记得连皮带肉吃更有营养。',
        '苹果香蕉都带着纤维小宝藏～',
        '水果富含可溶性纤维，果皮纤维更多，但要注意适量糖分摄入。'
      ),
      card(
        'fiber_square',
        5,
        '全谷物纤维',
        '燕麦、玉米、糙米这些粗粮比精米白面保留更多纤维，做成早餐粥既暖胃又养生。',
        '燕麦玉米是全谷物纤维的冠军！',
        '全谷物比精制谷物保留更多膳食纤维和 B 族维生素，建议替代部分主食。'
      ),
    ],
    quizzes: [
      quiz('fiber_square', 1, 'single_choice', '下面哪种食物含有丰富的膳食纤维？', ['糖果',
'西蓝花', '蛋糕', '汽水'], 1, '糖果蛋糕和汽水含糖高、纤维少，蔬菜才是纤维大户。'),
      quiz(
        'fiber_square',
        2,
        'pairing',
        '把食物和它的类别配对起来',
        ['西蓝花', '苹果', '燕麦', '蔬菜', '水果', '谷物'],
        [0, 1, 2],
        '西蓝花是蔬菜，苹果是水果，燕麦是谷物～'
      ),
      quiz(
        'fiber_square',
        3,
        'ordering',
        '膳食纤维的旅程，按正确顺序排列',
        ['到达大肠被居民吃掉', '吃进嘴巴', '一路下滑到肠道'],
        [1, 2, 0],
        '先吃进嘴，再滑到肠道，最后到大肠被益生菌吃掉。'
      ),
    ],
  },

  ferment_workshop: {
    module_code: 'ferment_workshop',
    name: '发酵工坊',
    description: '认识肠道里负责发酵的小工厂',
    cards: [
      card(
        'ferment_workshop',
        1,
        '发酵是什么？',
        '发酵是微生物把食物残渣分解成小分子的过程，就像工坊里的工人把原料加工成产品。',
```

```
        '发酵就是小工人们把食物加工成新东西～',
        '肠道菌群发酵不可消化碳水化合物，产生短链脂肪酸、气体等代谢产物。'
      ),
      card(
        'ferment_workshop',
        2,
        '肠道小居民',
        '益生菌是住在肠道里的好居民，它们帮助我们消化食物、保护健康，是发酵工坊的主力工人。',
        '益生菌是工坊里勤劳的好工人！',
        '双歧杆菌、乳酸杆菌等益生菌是肠道发酵的主力军。'
      ),
      card(
        'ferment_workshop',
        3,
        '发酵的产物',
        '发酵工坊会产出短链脂肪酸、维生素，还会产生一点点气体，这都是工坊正常工作的信号。',
        '工坊会产出营养和一点点小气泡～',
        '发酵产物包括短链脂肪酸、B 族维生素和气体，是肠道健康的指标。'
      ),
      card(
        'ferment_workshop',
        4,
        '酸奶里的益生菌',
        '酸奶、泡菜这些发酵食物里藏着活的益生菌，吃进肚子后能壮大工坊的工人队伍。',
        '酸奶里有活的益生菌小工人！',
        '发酵乳制品和发酵蔬菜含有活性益生菌，有助于维持肠道菌群。'
      ),
      card(
        'ferment_workshop',
        5,
        '帮助发酵的食物',
        '多吃膳食纤维，工坊的工人就有充足的原料，发酵得更起劲，产出更多好营养。',
        '多吃纤维，工坊工人干活更有劲！',
        '益生元（可发酵纤维）是益生菌的食物，能促进有益菌增殖。'
      ),
    ],
    quizzes: [
      quiz('ferment_workshop', 1, 'single_choice', '肠道里负责发酵工作的小居民是谁？', ['病
毒', '益生菌', '蚊虫', '小鱼'], 1, '益生菌就是工坊里勤劳的发酵工人。'),
      quiz('ferment_workshop', 2, 'pairing', '把发酵食物和它的类别配对', ['酸奶', '泡菜', '汽
水', '发酵乳', '发酵蔬菜', '高糖饮料'], [0, 1, 2], '酸奶是发酵乳，泡菜是发酵蔬菜，汽水不是发酵食
物。'),
      quiz('ferment_workshop', 3, 'ordering', '发酵工坊的工作流程，按顺序排列', ['产出短链脂肪
酸', '居民吃掉纤维原料', '原料到达大肠'], [2, 1, 0], '原料先到大肠，被居民吃掉，再产出短链脂肪
酸。'),
    ],
  },

  scfa_spring: {
    module_code: 'scfa_spring',
    name: '短链脂肪酸泉',
    description: '探索肠道居民产出的能量泉水',
    cards: [
      card(
        'scfa_spring',
```

```
        1,
        '短链脂肪酸是什么？',
        '短链脂肪酸是肠道益生菌发酵纤维后产出的「能量泉水」，能给肠道和身体提供能量。',
        '短链脂肪酸是益生菌产出的能量泉水！',
        '乙酸、丙酸、丁酸是主要的短链脂肪酸，具有供能和免疫调节作用。'
      ),
      card(
        'scfa_spring',
        2,
        '能量泉水从哪来？',
        '泉水不是凭空出现的，需要益生菌吃到足够的膳食纤维才能生产出来。',
        '泉水需要纤维原料才能生产出来～',
        '可发酵纤维是合成短链脂肪酸的原料，膳食纤维摄入不足则产量下降。'
      ),
      card(
        'scfa_spring',
        3,
        '滋养肠道屏障',
        '丁酸是城墙砖块最喜欢的能量，喝了泉水，肠道屏障的城墙就更结实。',
        '泉水能让肠道城墙更结实！',
        '丁酸是结肠上皮细胞的主要能量来源，能维护肠道屏障完整。'
      ),
      card(
        'scfa_spring',
        4,
        '哪些食物产泉多',
        '抗性淀粉（放凉的土豆）、菊粉（洋葱大蒜）、燕麦这些食物，都是产泉大户。',
        '燕麦土豆是产泉水的大户～',
        '抗性淀粉和菊粉等益生元能高效促进短链脂肪酸生成。'
      ),
      card(
        'scfa_spring',
        5,
        '如何让泉水更充沛',
        '规律三餐、多吃纤维、早睡早起，肠道居民状态好，产出的泉水就越多。',
        '吃好睡好，泉水就哗哗流～',
        '规律作息和均衡膳食能稳定肠道微生态，维持短链脂肪酸水平。'
      ),
    ],
    quizzes: [
      quiz('scfa_spring', 1, 'single_choice', '短链脂肪酸主要由谁产生？', ['胃', '肠道益生
菌', '肝脏', '心脏'], 1, '肠道益生菌发酵纤维，才会产出短链脂肪酸。'),
      quiz('scfa_spring', 2, 'single_choice', '哪种食物能促进短链脂肪酸生成？', ['奶茶', '薯
片', '燕麦', '汽水'], 2, '燕麦富含可发酵纤维，是产泉大户。'),
      quiz('scfa_spring', 3, 'ordering', '泉水诞生的过程，按顺序排列', ['产出能量泉水', '益生菌
吃到纤维', '喝下燕麦粥'], [2, 1, 0], '先吃进纤维，益生菌发酵，再产出短链脂肪酸。'),
    ],
  },

  barrier_wall: {
    module_code: 'barrier_wall',
    name: '肠道屏障城墙',
    description: '了解肠道屏障如何保护身体',
```

```
    cards: [
      card(
        'barrier_wall',
        1,
        '肠道屏障是什么？',
        '肠道屏障就像一座城墙，把营养物质放进来，把坏东西挡在外面，保护我们的身体。',
        '肠道屏障是保护身体的城墙！',
        '肠道屏障由机械屏障、化学屏障和免疫屏障组成，阻止有害物质进入循环系统。'
      ),
      card(
        'barrier_wall',
        2,
        '城墙的砖块',
        '肠道细胞像一块块砖紧紧挨着，砖块之间缝隙越小，城墙越牢固。',
        '肠道细胞是城墙的小砖块，挨得紧紧的！',
        '肠上皮细胞间的紧密连接是机械屏障的关键，控制物质通透性。'
      ),
      card(
        'barrier_wall',
        3,
        '粘液护城河',
        '城墙前还有一条粘液护城河，坏菌游不过来，好菌在护城河里安家。',
        '粘液护城河挡住坏菌去路～',
        '肠道粘液层含有抗菌肽和分泌型 IgA，是重要的化学屏障。'
      ),
      card(
        'barrier_wall',
        4,
        '城墙破了会怎样',
        '城墙出现裂缝，坏东西钻进来，肚子就会不舒服，还可能容易过敏感冒。',
        '城墙裂缝会让坏东西钻进来！',
        '肠道屏障受损（肠漏）与食物不耐受、过敏和慢性炎症相关。'
      ),
      card(
        'barrier_wall',
        5,
        '加固城墙的方法',
        '多吃膳食纤维和益生菌，补充能量泉水，规律作息，城墙就会又高又结实。',
        '纤维+益生菌+好睡眠，城墙更结实！',
        '膳食纤维、益生菌、充足的丁酸和良好作息共同维护肠道屏障。'
      ),
    ],
    quizzes: [
      quiz('barrier_wall', 1, 'single_choice', '肠道屏障的主要作用是什么？', ['消化食物', '挡住坏东西保护身体', '制造血液', '储存大便'], 1, '肠道屏障把坏东西挡在外面，保护身体。'),
      quiz('barrier_wall', 2, 'pairing', '把城墙的部件和它的作用配对', ['肠道细胞', '粘液', '益生菌', '紧密砖块', '护城河', '守卫居民'], [0, 1, 2], '肠道细胞是砖块，粘液是护城河，益生菌是守卫。'),
      quiz('barrier_wall', 3, 'ordering', '加固城墙，按正确顺序做', ['补充益生菌', '多喝能量泉水', '多吃膳食纤维'], [2, 0, 1], '先多吃纤维喂养居民，再补益生菌，喝足泉水让城墙更结实。'),
    ],
  },

  eco_station: {
```

```
    module_code: 'eco_station',
    name: '生态平衡观测站',
    description: '观测肠道菌群的生态平衡',
    cards: [
      card(
        'eco_station',
        1,
        '菌群生态',
        '肠道里住着数以万亿计的微生物，好菌和坏菌相互制衡，保持平衡身体才健康。',
        '肠道里有万亿个微生物小伙伴～',
        '肠道菌群由细菌、真菌等构成，好菌与坏菌的平衡影响整体健康。'
      ),
      card(
        'eco_station',
        2,
        '多样性很重要',
        '菌群的种类越丰富，生态越稳定，就像森林里植物多才不容易生病。',
        '菌群种类越多样，身体越强壮！',
        '菌群多样性是肠道健康的标志，多样性降低与多种疾病相关。'
      ),
      card(
        'eco_station',
        3,
        '破坏平衡的因素',
        '经常吃高糖食物、乱用抗生素、熬夜，都会让好菌减少、坏菌变多。',
        '高糖、乱吃药、熬夜会赶走好菌！',
        '高糖饮食、抗生素滥用和不规律作息会破坏肠道菌群平衡。'
      ),
      card(
        'eco_station',
        4,
        '维持平衡',
        '均衡饮食、规律作息、适量运动，就像给生态园浇水施肥，菌群更健康。',
        '均衡饮食是给菌群生态园浇水！',
        '饮食、作息和运动是塑造健康肠道菌群的核心生活方式因素。'
      ),
      card(
        'eco_station',
        5,
        '观测指标',
        '便便的形状、颜色，还有情绪和精力，都是观测菌群生态的小信号。',
        '便便和情绪都是生态观测的信号灯～',
        '便便形态（Bristol 分型）、消化感受和整体精力可反映菌群状态。'
      ),
    ],
    quizzes: [
      quiz('eco_station', 1, 'single_choice', '以下哪个做法最有利于肠道菌群平衡？', ['天天吃炸鸡', '均衡饮食多运动', '乱吃抗生素', '经常熬夜'], 1, '均衡饮食加规律运动能维持菌群生态平衡。'),
      quiz('eco_station', 2, 'pairing', '把行为和它对菌群的影响配对', ['高糖饮食', '规律作息', '多吃纤维', '赶走好菌', '稳定菌群', '喂养好菌'], [0, 1, 2], '高糖赶走好菌，规律作息稳定菌群，纤维喂养好菌。'),
      quiz('eco_station', 3, 'ordering', '发现菌群失衡后，按正确步骤调整', ['多吃膳食纤维', '观察便便信号', '恢复规律作息'], [1, 0, 2], '先观察信号，再吃纤维喂养好菌，最后作息跟上。'),
    ],
```

```
  },
}

export function findQuiz(questionId: string): QuizContent | null {
  for (const code of MODULE_ORDER) {
    const q = MODULE_DEFS[code].quizzes.find((x) => x.id === questionId)
    if (q) return q
  }
  return null
}
import type { FastifyInstance, FastifyReply } from 'fastify'
import { listModules, getCards, answerQuiz, recordVideoWatched } from './classroom.serv
ice.js'

function badRequest(reply: FastifyReply, message: string): FastifyReply {
  return reply.status(400).send({ code: 'VALIDATION', message })
}

export default async function classroomRoutes(fastify: FastifyInstance): Promise<void>
{
  fastify.get('/api/classroom/modules', async (req, reply) => {
    const { child_id } = req.query as { child_id?: string }
    const childId = Number(child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    return { code: 0, data: await listModules(childId) }
  })

  fastify.get('/api/classroom/modules/:code/cards', async (req) => {
    const { code } = req.params as { code: string }
    return { code: 0, data: await getCards(code) }
  })

  fastify.post('/api/classroom/modules/:code/watch', async (req, reply) => {
    const { code } = req.params as { code: string }
    const body = req.body as { child_id?: number }
    const childId = Number(body.child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    return { code: 0, data: await recordVideoWatched(childId, code) }
  })

  fastify.post('/api/classroom/quiz/answer', async (req, reply) => {
    const body = req.body as { child_id?: number; question_id?: string; answer?: unknow
n }
    const childId = Number(body.child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    if (!body.question_id) return badRequest(reply, 'question_id 必填')
    if (body.answer === undefined) return badRequest(reply, 'answer 必填')
    return { code: 0, data: await answerQuiz(childId, body.question_id, body.answer) }
  })
}
import { eq, and } from 'drizzle-orm'
import { db } from '../../db/index.js'
import { knowledgeModuleProgress, quizRecords } from '../../db/schema/index.js'
```

```
import { MODULE_ORDER, MODULE_DEFS, findQuiz, type ModuleCode, type QuizContent } from
'./classroom.content.js'
import { addGardenXp, todayLocal } from '../garden/garden.service.js'
import { onQuizEvent } from '../badges/badge-hooks.js'
import { throwError } from '../../config/errors'

const QUIZ_XP = 3
const WATCH_XP = 5

export async function listModules(childId: number) {
  if (!childId) throwError('CHILD_001')
  const progress = await db.select().from(knowledgeModuleProgress).where(eq(knowledgeMo
duleProgress.childId, childId))
  const byCode = new Map(progress.map((p) => [p.moduleCode, p]))

  return MODULE_ORDER.map((code) => {
    const def = MODULE_DEFS[code]
    const p = byCode.get(code)
    return {
      module_code: code,
      name: def.name,
      description: def.description,
      cards_unlocked: p?.cardsUnlocked ?? 0,
      cards_total: def.cards.length,
      quizzes_passed: p?.quizzesPassed ?? 0,
      quizzes_total: def.quizzes.length,
      animation_watched: p?.animationWatched ?? false,
      completed: Boolean(p?.completedAt),
    }
  })
}

export async function recordVideoWatched(childId: number, moduleCode: string) {
  if (!childId) throwError('CHILD_001')
  const def = MODULE_DEFS[moduleCode as ModuleCode]
  if (!def) throwError('CLASSROOM_001')

  const progress = await getOrCreateProgress(childId, moduleCode as ModuleCode)

  let xpGained = 0
  if (!progress.animationWatched) {
    const cardsUnlocked = Math.max(progress.cardsUnlocked, 1)
    await db
      .update(knowledgeModuleProgress)
      .set({ animationWatched: true, cardsUnlocked })
      .where(eq(knowledgeModuleProgress.id, progress.id))
    xpGained = WATCH_XP
    await addGardenXp(childId, WATCH_XP)
  }

  const [updated] = await db
    .select()
```

```
    .from(knowledgeModuleProgress)
    .where(eq(knowledgeModuleProgress.id, progress.id))

  return {
    module_code: moduleCode,
    animation_watched: updated.animationWatched,
    cards_unlocked: updated.cardsUnlocked,
    cards_total: def.cards.length,
    quizzes_passed: updated.quizzesPassed,
    xp_gained: xpGained,
  }
}

export async function getCards(moduleCode: string) {
  const def = MODULE_DEFS[moduleCode as ModuleCode]
  if (!def) throwError('CLASSROOM_001')
  return def.cards.map((c) => ({
    id: c.id,
    module_code: def.module_code,
    title: c.title,
    front_image: c.front_image,
    back_content: c.back_content,
    child_summary: c.child_summary,
    parent_detail: c.parent_detail,
  }))
}

function normalizeAnswer(answer: unknown): number[] {
  if (Array.isArray(answer)) return answer.map(Number)
  return [Number(answer)]
}

function isCorrect(q: QuizContent, answer: unknown): boolean {
  const expected = Array.isArray(q.answer) ? q.answer.map(Number) : [Number(q.answer)]
  const given = normalizeAnswer(answer)
  return JSON.stringify(expected) === JSON.stringify(given)
}

async function getOrCreateProgress(childId: number, moduleCode: ModuleCode) {
  let [row] = await db
    .select()
    .from(knowledgeModuleProgress)
    .where(and(eq(knowledgeModuleProgress.childId, childId), eq(knowledgeModuleProgres
s.moduleCode, moduleCode)))
  if (!row) {
    const [created] = await db
      .insert(knowledgeModuleProgress)
      .values({ childId, moduleCode, cardsTotal: MODULE_DEFS[moduleCode].cards.length
})
      .returning()
    row = created
  }
```

```
  return row
}

export async function answerQuiz(childId: number, questionId: string, answer: unknown)
{
  if (!childId) throwError('CHILD_001')
  const q = findQuiz(questionId)
  if (!q) throwError('CLASSROOM_002')

  const correct = isCorrect(q, answer)
  const correctAnswer = Array.isArray(q.answer) ? q.answer : Number(q.answer)

  await db.insert(quizRecords).values({
    childId,
    quizDate: todayLocal(),
    moduleCode: q.module_code,
    questionType: q.type,
    question: q.question,
    answerCorrect: correct,
  })

  let xpGained = 0
  let moduleCompleted = false
  let quizzesPassed = 0

  if (correct) {
    const progress = await getOrCreateProgress(childId, q.module_code)
    quizzesPassed = progress.quizzesPassed + 1
    const total = MODULE_DEFS[q.module_code].quizzes.length
    moduleCompleted = quizzesPassed >= total

    await db
      .update(knowledgeModuleProgress)
      .set({
        quizzesPassed: quizzesPassed,
        completedAt: moduleCompleted ? new Date() : progress.completedAt,
      })
      .where(eq(knowledgeModuleProgress.id, progress.id))

    xpGained = QUIZ_XP
    await addGardenXp(childId, QUIZ_XP)
  }

  const badges = await onQuizEvent(childId)
  return {
    correct,
    xp_gained: xpGained,
    correct_answer: correctAnswer,
    answer_hint: correct ? null : q.answer_hint,
    module_completed: moduleCompleted,
    quizzes_passed: quizzesPassed,
```

```
    badges_awarded: badges,
  }
}
import type { FastifyInstance } from 'fastify'
import { getGardenState, logAction, todayInteractionCount, ensureChildExists } from './
garden.service.js'
import { evaluate } from '../badges/engine.js'

export default async function gardenRoutes(fastify: FastifyInstance): Promise<void> {
  fastify.get('/api/garden/state', async (req) => {
    const { child_id } = req.query as { child_id?: string }
    const childId = Number(child_id)
    if (!childId) return { code: 'VALIDATION', message: 'child_id 必填' }
    const data = await getGardenState(childId)
    return { code: 0, data }
  })

  fastify.get('/api/garden/actions/today-count', async (req) => {
    const { child_id } = req.query as { child_id?: string }
    const childId = Number(child_id)
    if (!childId) return { code: 'VALIDATION', message: 'child_id 必填' }
    const count = await todayInteractionCount(childId)
    return { code: 0, data: { child_id: childId, interaction_count_today: count } }
  })

  fastify.post('/api/garden/log-action', async (req) => {
    const body = req.body as { child_id?: number; action_type?: string; action_detail?:
Record<string, unknown> }
    const childId = Number(body.child_id)
    if (!childId) return { code: 'VALIDATION', message: 'child_id 必填' }
    const actionType = body.action_type
    if (!actionType || !['feed', 'explore', 'magnifier', 'treatment'].includes(actionTy
pe)) {
      return { code: 'VALIDATION', message: 'action_type 无效' }
    }
    await ensureChildExists(childId)
    const data = await logAction({ childId, actionType: actionType as never, actionDeta
il: body.action_detail })
    const badges = await evaluate(childId)
    return { code: 0, data: { ...data, badges_awarded: badges } }
  })
}
import { eq, and, sql } from 'drizzle-orm'
import { db } from '../../db/index.js'
import { gardenStates, gardenActionLogs, children } from '../../db/schema/index.js'
import { collectStats, evaluateStage } from './stage.service.js'
import { throwError } from '../../config/errors'

const HEALTHY_FOODS = new Set(['broccoli', 'carrot', 'yogurt', 'apple', 'corn'])
const HIGH_SUGAR_FOODS = new Set(['candy', 'cake'])

const FEED_XP = 2
const FEED_MOISTURE_UP = 10
const HIGH_SUGAR_MOISTURE_DOWN = 8
```

```
const WATER_MOISTURE_UP = 15

export type ActionType = 'feed' | 'explore' | 'magnifier' | 'treatment'

export interface LogActionInput {
  childId: number
  actionType: ActionType
  actionDetail?: Record<string, unknown> | null
}

function fmtLocalDate(d: Date): string {
  const y = d.getFullYear()
  const m = String(d.getMonth() + 1).padStart(2, '0')
  const day = String(d.getDate()).padStart(2, '0')
  return `${y}-${m}-${day}`
}

export function todayLocal(): string {
  return fmtLocalDate(new Date())
}

async function computeAndSyncStage(childId: number, currentStage: number): Promise<Retu
rnType<typeof evaluateStage>> {
  const stats = await collectStats(childId)
  const stage = evaluateStage(stats)
  if (stage.growthStage !== currentStage) {
    await db
      .update(gardenStates)
      .set({ growthStage: stage.growthStage, lastUpdated: new Date() })
      .where(eq(gardenStates.childId, childId))
  }
  return stage
}

export async function getGardenState(childId: number) {
  const row = (await db.select().from(gardenStates).where(eq(gardenStates.childId, chil
dId)))[0]
  if (!row) throwError('GARDEN_001')

  const interactionCount = await todayInteractionCount(childId)
  const stage = await computeAndSyncStage(childId, row.growthStage)
  const storedUnlocked = (row.unlockedFeatures as string[]) || []
  const unlocked = Array.from(new Set([...storedUnlocked, ...stage.unlocked]))

  return {
    child_id: childId,
    current_state: row.currentState,
    moisture_level: row.moistureLevel,
    growth_stage: stage.growthStage,
    garden_xp: row.gardenXp,
    interaction_count: interactionCount,
    unlocked_features: unlocked,
```

```
    stage_label: stage.label,
  }
}

export async function todayInteractionCount(childId: number): Promise<number> {
  const rows = await db
    .select({ id: gardenActionLogs.id })
    .from(gardenActionLogs)
    .where(and(eq(gardenActionLogs.childId, childId), sql`date(${gardenActionLogs.creat
edAt}) = ${todayLocal()}`))

  return rows.length
}

function applyFoodEffect(state: string, moisture: number, foodType: string): { currentS
tate: string; moistureLevel: number } {
  let nextState = state
  let nextMoisture = moisture

  if (HIGH_SUGAR_FOODS.has(foodType)) {
    nextMoisture = Math.max(0, moisture - HIGH_SUGAR_MOISTURE_DOWN)
    nextState = 'high_sugar'
  } else if (HEALTHY_FOODS.has(foodType)) {
    nextMoisture = Math.min(100, moisture + FEED_MOISTURE_UP)
    if (state === 'high_sugar' || state === 'dry') nextState = 'recovering'
    else nextState = 'healthy'
    if (nextState === 'recovering' && nextMoisture >= 60) nextState = 'healthy'
  }

  return { currentState: nextState, moistureLevel: nextMoisture }
}

export async function logAction(input: LogActionInput) {
  const existing = (await db.select().from(gardenStates).where(eq(gardenStates.childId,
input.childId)))[0]
  if (!existing) throwError('GARDEN_001')

  const { actionType, actionDetail } = input
  let currentState = existing.currentState
  let moistureLevel = existing.moistureLevel
  let xpGain = 0

  if (actionType === 'feed') {
    const foodType = String(actionDetail?.food_type || '')
    const effect = applyFoodEffect(currentState, moistureLevel, foodType)
    currentState = effect.currentState
    moistureLevel = effect.moistureLevel
    xpGain = FEED_XP
  } else if (actionType === 'treatment') {
    const treatment = String(actionDetail?.treatment || '')
    if (treatment === 'water') {
      moistureLevel = Math.min(100, moistureLevel + WATER_MOISTURE_UP)
      currentState = currentState === 'dry' || currentState === 'high_sugar' ? 'recover
ing' : currentState
```

```
      if (moistureLevel >= 60) currentState = 'healthy'
    }
  }

  await db.transaction(async (tx) => {
    await tx
      .update(gardenStates)
      .set({ currentState, moistureLevel, gardenXp: existing.gardenXp + xpGain, lastUpd
ated: new Date() })
      .where(eq(gardenStates.childId, input.childId))

    await tx.insert(gardenActionLogs).values({
      childId: input.childId,
      actionType,
      actionDetail: actionDetail ?? null,
    })
  })

  const interactionCount = await todayInteractionCount(input.childId)
  const stage = await computeAndSyncStage(input.childId, existing.growthStage)

  return {
    child_id: input.childId,
    current_state: currentState,
    moisture_level: moistureLevel,
    growth_stage: stage.growthStage,
    garden_xp: existing.gardenXp + xpGain,
    xp_gained: xpGain,
    interaction_count: interactionCount,
    stage_label: stage.label,
  }
}

export async function ensureChildExists(childId: number): Promise<void> {
  const row = (await db.select().from(children).where(eq(children.id, childId)))[0]
  if (!row) throwError('CHILD_001')
}

export async function addGardenXp(childId: number, amount: number): Promise<number> {
  const [row] = await db
    .update(gardenStates)
    .set({ gardenXp: sql`${gardenStates.gardenXp} + ${amount}`, lastUpdated: new Date()
})
    .where(eq(gardenStates.childId, childId))
    .returning({ gardenXp: gardenStates.gardenXp })
  return row?.gardenXp ?? 0
}
import { count, eq } from 'drizzle-orm'
import { db } from '../../db/index.js'
import { gardenActionLogs, checkinCalendar, badgeAwards } from '../../db/schema/index.j
s'

export interface GardenStats {
```

```
  checkinDays: number
  feedCount: number
  badgeCount: number
}

export interface StageInfo {
  growthStage: number
  label: string
  unlocked: string[]
}

export const STAGE_REQS: { stage: number; checkinDays: number; feedCount: number; badge
Count: number; label: string; unlock: string }[] = [
  { stage: 1, checkinDays: 0, feedCount: 0, badgeCount: 0, label: '种子', unlock: '' },
  { stage: 2, checkinDays: 3, feedCount: 10, badgeCount: 0, label: '幼苗', unlock: '角色
动画' },
  { stage: 3, checkinDays: 7, feedCount: 30, badgeCount: 0, label: '成长', unlock: '放大
镜' },
  { stage: 4, checkinDays: 21, feedCount: 100, badgeCount: 3, label: '丰收', unlock: '花
园状态切换' },
  { stage: 5, checkinDays: 50, feedCount: 200, badgeCount: 6, label: '大师', unlock: '全
角色 + 全部知识模块' },
  { stage: 6, checkinDays: 100, feedCount: 500, badgeCount: 10, label: '终极', unlock:
'花园完全体（金边特效）' },
]

export async function collectStats(childId: number): Promise<GardenStats> {
  const [cal] = await db
    .select({ value: count() })
    .from(checkinCalendar)
    .where(eq(checkinCalendar.childId, childId))
  const [feed] = await db
    .select({ value: count() })
    .from(gardenActionLogs)
    .where(eq(gardenActionLogs.childId, childId))
  const [badge] = await db
    .select({ value: count() })
    .from(badgeAwards)
    .where(eq(badgeAwards.childId, childId))

  return {
    checkinDays: Number(cal?.value) || 0,
    feedCount: Number(feed?.value) || 0,
    badgeCount: Number(badge?.value) || 0,
  }
}

export function evaluateStage(stats: GardenStats): StageInfo {
  let stage = 1
  for (const req of STAGE_REQS) {
    if (stats.checkinDays >= req.checkinDays && stats.feedCount >= req.feedCount && sta
ts.badgeCount >= req.badgeCount) {
      stage = req.stage
    } else {
      break
    }
  }
```

```
  const unlocked = STAGE_REQS.filter((r) => r.stage <= stage).map((r) => r.unlock).filt
er(Boolean)
  return { growthStage: stage, label: STAGE_REQS[stage - 1].label, unlocked }
}
import type { FastifyInstance, FastifyReply } from 'fastify'
import { requireParentId } from '../auth/auth.routes.js'
import { generateReport, weekRange, monthRange } from './report.service.js'

function badRequest(reply: FastifyReply, message: string): FastifyReply {
  return reply.status(400).send({ code: 'VALIDATION', message })
}

function isEmpty(r: Record<string, unknown>): boolean {
  const key = r as {
    checkin_rate: number
    stool_count: number
    feed_count: number
    quiz_accuracy: number
    badges: { total: number }
  }
  return key.checkin_rate === 0 && key.stool_count === 0 && key.feed_count === 0 && ke
y.quiz_accuracy === 0 && key.badges.total === 0
}

export default async function reportRoutes(fastify: FastifyInstance): Promise<void> {
  fastify.get('/api/report/weekly', async (req, reply) => {
    requireParentId(req)
    const { child_id, week } = req.query as { child_id?: string; week?: string }
    const childId = Number(child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    const data = await generateReport(childId, 'week', weekRange(week))
    return { code: 0, data: isEmpty(data as never) ? null : data }
  })

  fastify.get('/api/report/monthly', async (req, reply) => {
    requireParentId(req)
    const { child_id, month } = req.query as { child_id?: string; month?: string }
    const childId = Number(child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    const data = await generateReport(childId, 'month', monthRange(month))
    return { code: 0, data: isEmpty(data as never) ? null : data }
  })
}
import { and, eq, gte, lte, inArray, sql } from 'drizzle-orm'
import { db } from '../../db/index.js'
import {
  checkinCalendar,
  gardenStates,
  gardenActionLogs,
  badgeAwards,
  stoolAnalyses,
  quizRecords,
```

```
  knowledgeModuleProgress,
  checkinRecords,
} from '../../db/schema/index.js'
import { throwError } from '../../config/errors'

const SUB_COLUMNS = ['subWater', 'subVegetable', 'subFruit', 'subOutdoor', 'subEarlySle
ep'] as const
const STAGE_LABELS = ['', '种子', '幼苗', '成长', '丰收', '大师', '终极']

function fmtDate(d: Date): string {
  const y = d.getFullYear()
  const m = String(d.getMonth() + 1).padStart(2, '0')
  const day = String(d.getDate()).padStart(2, '0')
  return `${y}-${m}-${day}`
}

function addDays(dateStr: string, n: number): string {
  const d = new Date(dateStr)
  d.setDate(d.getDate() + n)
  return fmtDate(d)
}

export function weekRange(dateStr?: string): { start: string; end: string } {
  const ref = dateStr ? new Date(dateStr) : new Date()
  const day = ref.getDay() || 7
  const monday = new Date(ref)
  monday.setDate(ref.getDate() - day + 1)
  const start = fmtDate(monday)
  return { start, end: addDays(start, 6) }
}

export function monthRange(month?: string): { start: string; end: string } {
  const ref = month ? new Date(`${month}-01T00:00:00`) : new Date()
  const start = fmtDate(new Date(ref.getFullYear(), ref.getMonth(), 1))
  const end = fmtDate(new Date(ref.getFullYear(), ref.getMonth() + 1, 0))
  return { start, end }
}

function longestRun(sortedDates: string[]): number {
  if (sortedDates.length === 0) return 0
  let longest = 1
  let run = 1
  for (let i = 1; i < sortedDates.length; i++) {
    const prev = new Date(sortedDates[i - 1])
    const cur = new Date(sortedDates[i])
    prev.setDate(prev.getDate() + 1)
    run = prev.getTime() === cur.getTime() ? run + 1 : 1
    if (run > longest) longest = run
  }
  return longest
}
```

```
function daysInPeriod(start: string, end: string): number {
  const a = new Date(start)
  const b = new Date(end)
  return Math.round((b.getTime() - a.getTime()) / 86_400_000) + 1
}

export async function generateReport(childId: number, periodType: 'week' | 'month' | 'd
ay', range: { start: string; end: string }) {
  if (!childId) throwError('CHILD_001')
  const { start, end } = range

  const calRows = await db
    .select({ calendarDate: checkinCalendar.calendarDate })
    .from(checkinCalendar)
    .where(
      and(
        eq(checkinCalendar.childId, childId),
        inArray(checkinCalendar.status, ['done', 'makeup']),
        gte(checkinCalendar.calendarDate, start),
        lte(checkinCalendar.calendarDate, end)
      )
    )
  const checkinDays = calRows.length
  const checkinRate = Math.round((checkinDays / daysInPeriod(start, end)) * 100)

  const allCal = await db
    .select({ calendarDate: checkinCalendar.calendarDate })
    .from(checkinCalendar)
    .where(and(eq(checkinCalendar.childId, childId), inArray(checkinCalendar.status,
['done', 'makeup']), lte(checkinCalendar.calendarDate, end)))
  const maxStreak = longestRun(allCal.map((r) => r.calendarDate).sort())

  const [garden] = await db.select().from(gardenStates).where(eq(gardenStates.childId,
childId))
  const growthStage = garden?.growthStage ?? 0
  const stageLabel = STAGE_LABELS[growthStage] ?? ''

  const badges = await db.select({ rarity: badgeAwards.rarity }).from(badgeAwards).wher
e(eq(badgeAwards.childId, childId))
  const badgeDist = { total: badges.length, bronze: 0, silver: 0, gold: 0 }
  for (const b of badges) badgeDist[b.rarity as 'bronze'] += 1

  const stools = await db
    .select({ bristolType: stoolAnalyses.bristolType, uploadedAt: stoolAnalyses.uploade
dAt })
    .from(stoolAnalyses)
    .where(and(eq(stoolAnalyses.childId, childId), sql`date(${stoolAnalyses.uploadedA
t}) between ${start} and ${end}`))
  const bristolDist: { bristol: number; count: number }[] = []
  const bristolMap = new Map<number, number>()
  for (const s of stools) {
    if (!s.bristolType) continue
    bristolMap.set(s.bristolType, (bristolMap.get(s.bristolType) ?? 0) + 1)
  }
  for (const [bristol, count] of Array.from(bristolMap.entries()).sort((a, b2) => a[0]
- b2[0])) {
```

```
    bristolDist.push({ bristol, count })
  }

  const feeds = await db
    .select({ id: gardenActionLogs.id })
    .from(gardenActionLogs)
    .where(and(eq(gardenActionLogs.childId, childId), eq(gardenActionLogs.actionType,
'feed'), sql`date(${gardenActionLogs.createdAt}) between ${start} and ${end}`))
  const feedCount = feeds.length

  const quizzes = await db
    .select({ answerCorrect: quizRecords.answerCorrect, quizDate: quizRecords.quizDate
})
    .from(quizRecords)
    .where(and(eq(quizRecords.childId, childId), gte(quizRecords.quizDate, start), lte
(quizRecords.quizDate, end)))
  const quizAccuracy = quizzes.length ? Math.round((quizzes.filter((q) => q.answerCorre
ct).length / quizzes.length) * 100) : 0

  const mods = await db.select({ completedAt: knowledgeModuleProgress.completedAt }).fr
om(knowledgeModuleProgress).where(eq(knowledgeModuleProgress.childId, childId))
  const modulesCompleted = mods.filter((m) => m.completedAt).length

  const subRecords = await db
    .select()
    .from(checkinRecords)
    .where(and(eq(checkinRecords.childId, childId), gte(checkinRecords.checkinDate, sta
rt), lte(checkinRecords.checkinDate, end)))
  let subDone = 0
  let subTotal = 0
  for (const r of subRecords) {
    for (const col of SUB_COLUMNS) {
      subTotal += 1
      if (r[col]) subDone += 1
    }
  }
  const subItemRate = subTotal ? Math.round((subDone / subTotal) * 100) : 0

  const activeDays = new Set<string>([
    ...calRows.map((r) => r.calendarDate),
    ...stools.map((s) => s.uploadedAt.toISOString().slice(0, 10)),
    ...quizzes.map((q) => String(q.quizDate)),
    ...subRecords.map((r) => r.checkinDate),
  ]).size

  // 花园状态分布：以周期内行为推断（健康食物→healthy / 高糖→high_sugar / 浇水→dry 恢复中）
  const actions = await db
    .select({ actionType: gardenActionLogs.actionType, actionDetail: gardenActionLogs.a
ctionDetail })
    .from(gardenActionLogs)
    .where(
      and(
        eq(gardenActionLogs.childId, childId),
        inArray(gardenActionLogs.actionType, ['feed', 'treatment']),
        sql`date(${gardenActionLogs.createdAt}) between ${start} and ${end}`
      )
    )
```

```
  const stateCounts: Record<string, number> = { healthy: 0, high_sugar: 0, dry: 0 }
  for (const a of actions) {
    if (a.actionType === 'feed') {
      const food = String((a.actionDetail as { food_type?: string } | null)?.food_type
?? '')
      if (food === 'candy' || food === 'cake') stateCounts.high_sugar += 1
      else stateCounts.healthy += 1
    } else if (a.actionType === 'treatment') {
      stateCounts.dry += 1
    }
  }
  const gardenStateDist = Object.entries(stateCounts).map(([state, count]) => ({ state,
count }))

  return {
    period: { start, end, type: periodType },
    checkin_rate: checkinRate,
    max_streak: maxStreak,
    growth_stage: growthStage,
    stage_label: stageLabel,
    badges: badgeDist,
    stool_count: stools.length,
    bristol_distribution: bristolDist,
    feed_count: feedCount,
    quiz_accuracy: quizAccuracy,
    modules_completed: modulesCompleted,
    sub_item_rate: subItemRate,
    active_days: activeDays,
    garden_state_distribution: gardenStateDist,
  }
}
import fs from 'node:fs'
import { env } from '../../config/env'
import { throwError } from '../../config/errors'
import { BRISTOL_PRESETS } from './stool.presets'

export interface AnalysisResult {
  bristol_type: number
  diagnosis: string
  task_suggestion: string
  is_valid: boolean
}

const TIMEOUT_MS = 30_000

/**
 * 照片分析客户端。
 * - 配置了 STOOL_API_KEY + STOOL_API_URL → 调外部 API（兼容 OpenAI 风格响应，字段映射见下）
 * - 未配置（默认）→ 本地规则模拟，保证开发环境可用；STOOL_MOCK=false 时依然返回模拟结果（无真实 AP
I 兜底）
 */
export async function analyzeStoolPhoto(imagePath: string): Promise<AnalysisResult> {
  if (env.stoolApiKey && env.stoolApiUrl) {
```

```
    try {
      return await callExternalApi(imagePath)
    } catch {
      throwError('STOOL_003')
    }
  }
  return mockAnalyze(imagePath)
}

async function callExternalApi(imagePath: string): Promise<AnalysisResult> {
  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), TIMEOUT_MS)
  try {
    const data = fs.readFileSync(imagePath)
    const form = new FormData()
    form.append('image', new Blob([data]), 'stool.jpg')
    const res = await fetch(env.stoolApiUrl!, {
      method: 'POST',
      headers: { Authorization: `Bearer ${env.stoolApiKey}` },
      body: form,
      signal: controller.signal,
    })
    if (!res.ok) throw new Error(`stool api status ${res.status}`)
    const json = (await res.json()) as {
      bristol_type?: number
      bristol?: number
      diagnosis?: string
      suggestion?: string
      task_suggestion?: string
      is_valid?: boolean
      valid?: boolean
    }
    const bristol = Number(json.bristol_type ?? json.bristol)
    if (!bristol || bristol < 1 || bristol > 7) throw new Error('invalid bristol type')
    const preset = BRISTOL_PRESETS[bristol]
    const valid = json.is_valid ?? json.valid ?? true
    return {
      bristol_type: bristol,
      diagnosis: json.diagnosis || preset.diagnosis,
      task_suggestion: json.task_suggestion || json.suggestion || preset.task_suggestio
n,
      is_valid: valid,
    }
  } finally {
    clearTimeout(timer)
  }
}

/** 本地模拟：按文件大小确定性生成一个 Bristol 类型，方便联调不同形态 */
function mockAnalyze(imagePath: string): AnalysisResult {
  const size = fs.statSync(imagePath).size
```

```
  const bristol = (size % 7) + 1
  const preset = BRISTOL_PRESETS[bristol]
  return { bristol_type: bristol, diagnosis: preset.diagnosis, task_suggestion: preset.
task_suggestion, is_valid: true }
}
export interface BristolPreset {
  bristol_type: number
  diagnosis: string
  task_suggestion: string
}

export const BRISTOL_PRESETS: Record<number, { diagnosis: string; task_suggestion: stri
ng }> = {
  1: { diagnosis: '兔子便便 — 有点干哦', task_suggestion: '多喝水，多吃纤维丰富的蔬菜～' },
  2: { diagnosis: '香肠便便 — 有点干硬', task_suggestion: '多喝水，多吃蔬果，适量运动～' },
  3: { diagnosis: '条状便便 — 基本正常', task_suggestion: '继续均衡饮食，注意补充水分～' },
  4: { diagnosis: '香蕉便 — 非常健康！', task_suggestion: '继续保持均衡饮食～' },
  5: { diagnosis: '软块便便 — 基本正常', task_suggestion: '注意多喝水，多吃蔬菜哦～' },
  6: { diagnosis: '糊状便便 — 需要关注', task_suggestion: '建议调整饮食，多吃纤维食物，多喝水～'
},
  7: { diagnosis: '水样便便 — 需要关注', task_suggestion: '肚子不舒服吗？记得补充水分，如果持续
要告诉爸爸妈妈哦～' },
}

export const ICON_TO_BRISTOL: Record<string, number> = {
  type_1_rabbit: 1,
  type_2_grape: 2,
  type_3_corn: 3,
  type_4_banana: 4,
  type_5_icecream: 5,
  type_6_marshmallow: 6,
  type_7_water: 7,
}
import type { FastifyInstance, FastifyReply } from 'fastify'
import { throwError } from '../../config/errors'
import { requireParentId } from '../auth/auth.routes.js'
import { selectIcon, uploadPhoto, getAnalysis, getLatest, stoolFilePath, fileExists, cr
eateReadStream, mimeOf } from './stool.service.js'

function badRequest(reply: FastifyReply, message: string): FastifyReply {
  return reply.status(400).send({ code: 'VALIDATION', message })
}

export default async function stoolRoutes(fastify: FastifyInstance): Promise<void> {
  fastify.post('/api/stool/select-icon', async (req, reply) => {
    const body = req.body as { child_id?: number; stool_icon_type?: string; bristol_typ
e?: number }
    const childId = Number(body.child_id)
    if (!childId) return badRequest(reply, 'child_id 必填')
    const data = await selectIcon({ child_id: childId, stool_icon_type: body.stool_icon
_type, bristol_type: body.bristol_type })
    return { code: 0, data }
  })

  fastify.post('/api/stool/upload', async (req, reply) => {
    if (!req.user?.parent_id) throwError('STOOL_005')
    let childId: number | undefined
```

```
    let file: { filename: string; mimetype: string; data: Buffer } | undefined

    for await (const part of req.parts()) {
      if (part.type === 'field') {
        if (part.fieldname === 'child_id') childId = Number(part.value)
      } else if (part.type === 'file') {
        file = { filename: part.filename, mimetype: part.mimetype, data: await part.toB
uffer() }
      }
    }
    if (!childId) return badRequest(reply, 'child_id 必填')
    if (!file) return badRequest(reply, '请上传图片文件')

    const data = await uploadPhoto(childId, file)
    return { code: 0, data }
  })

  fastify.get('/api/stool/analysis/:id', async (req) => {
    requireParentId(req)
    const { id } = req.params as { id: string }
    const data = await getAnalysis(Number(id))
    return { code: 0, data }
  })

  fastify.get('/api/stool/latest', async (req) => {
    requireParentId(req)
    const { child_id } = req.query as { child_id?: string }
    const childId = Number(child_id)
    if (!childId) return { code: 'VALIDATION', message: 'child_id 必填' }
    const data = await getLatest(childId)
    return { code: 0, data }
  })

  fastify.get('/uploads/*', async (req, reply) => {
    const wildcard = (req.params as { '*': string })['*']
    const filePath = stoolFilePath(wildcard)
    if (!fileExists(filePath)) return reply.status(404).send({ code: 'NOT_FOUND', messa
ge: '文件不存在' })
    return reply.type(mimeOf(wildcard)).send(createReadStream(filePath))
  })
}
import { mkdirSync, writeFileSync, existsSync, createReadStream } from 'node:fs'
import path from 'node:path'
import { fileURLToPath } from 'node:url'
import { eq, desc, and, sql } from 'drizzle-orm'
import { db } from '../../db/index.js'
import { stoolAnalyses } from '../../db/schema/index.js'
import { BRISTOL_PRESETS, ICON_TO_BRISTOL } from './stool.presets'
import { analyzeStoolPhoto } from './stool-analysis.client'
import { onStoolEvent } from '../badges/badge-hooks.js'
import { throwError } from '../../config/errors'
```

```
const __dirname = path.dirname(fileURLToPath(import.meta.url))
export const UPLOAD_DIR = path.resolve(__dirname, '../../../uploads')

const VALID_EXT = new Set(['.jpg', '.jpeg', '.png', '.webp'])
const MAX_SIZE = 10 * 1024 * 1024

function addDays(d: Date, days: number): Date {
  const copy = new Date(d)
  copy.setDate(copy.getDate() + days)
  return copy
}

export interface SelectIconInput {
  child_id: number
  stool_icon_type?: string
  bristol_type?: number
}

export async function selectIcon(input: SelectIconInput) {
  const childId = Number(input.child_id)
  if (!childId) throwError('CHILD_001')

  let bristol = Number(input.bristol_type)
  if (!bristol && input.stool_icon_type) bristol = ICON_TO_BRISTOL[input.stool_icon_typ
e] ?? 0
  if (!bristol || bristol < 1 || bristol > 7) bristol = 4

  const preset = BRISTOL_PRESETS[bristol]
  const now = new Date()
  const [row] = await db
    .insert(stoolAnalyses)
    .values({
      childId,
      mode: 'icon_selection',
      stoolIconType: input.stool_icon_type || null,
      bristolType: bristol,
      diagnosis: preset.diagnosis,
      taskSuggestion: preset.task_suggestion,
      uploadedAt: now,
      expiresAt: addDays(now, 3),
    })
    .returning()

  const badges = await onStoolEvent(childId)
  return {
    analysis_id: row.id,
    mode: 'icon_selection',
    bristol_type: bristol,
    diagnosis: preset.diagnosis,
    task_suggestion: preset.task_suggestion,
    badges_awarded: badges,
```

```
  }
}

export interface UploadFile {
  filename: string
  mimetype: string
  data: Buffer
}

export async function uploadPhoto(childId: number, file: UploadFile) {
  if (!childId) throwError('CHILD_001')
  const ext = path.extname(file.filename || '').toLowerCase()
  if (!VALID_EXT.has(ext) || !file.data || file.data.length > MAX_SIZE) throwError('STO
OL_004')

  mkdirSync(UPLOAD_DIR, { recursive: true })
  const filename = `${childId}-${Date.now()}${ext}`
  const filePath = path.join(UPLOAD_DIR, filename)
  writeFileSync(filePath, file.data)

  const result = await analyzeStoolPhoto(filePath)
  if (!result.is_valid) throwError('STOOL_001')

  const now = new Date()
  const [row] = await db
    .insert(stoolAnalyses)
    .values({
      childId,
      mode: 'photo_upload',
      imageUrl: `/uploads/${filename}`,
      bristolType: result.bristol_type,
      diagnosis: result.diagnosis,
      taskSuggestion: result.task_suggestion,
      isValid: true,
      uploadedAt: now,
      expiresAt: addDays(now, 3),
    })
    .returning()

  const badges = await onStoolEvent(childId)
  return {
    analysis_id: row.id,
    mode: 'photo_upload',
    bristol_type: result.bristol_type,
    diagnosis: result.diagnosis,
    task_suggestion: result.task_suggestion,
    is_valid: true,
    image_url: row.imageUrl,
    badges_awarded: badges,
  }
}
```

```
function toDTO(row: typeof stoolAnalyses.$inferSelect) {
  return {
    analysis_id: row.id,
    mode: row.mode,
    bristol_type: row.bristolType,
    diagnosis: row.diagnosis,
    task_suggestion: row.taskSuggestion,
    image_url: row.imageUrl,
    is_valid: row.isValid,
    uploaded_at: row.uploadedAt,
    expires_at: row.expiresAt,
  }
}

export async function getAnalysis(id: number) {
  const [row] = await db.select().from(stoolAnalyses).where(eq(stoolAnalyses.id, id))
  if (!row) throwError('STOOL_002')
  return toDTO(row)
}

export async function getLatest(childId: number) {
  if (!childId) throwError('CHILD_001')
  const [row] = await db
    .select()
    .from(stoolAnalyses)
    .where(eq(stoolAnalyses.childId, childId))
    .orderBy(desc(stoolAnalyses.id))
    .limit(1)
  if (!row) throwError('STOOL_002')
  return toDTO(row)
}

/** 打卡页便便联动：最近一条未过期（3 天有效）的分析 */
export async function getRecentStool(childId: number) {
  const [row] = await db
    .select()
    .from(stoolAnalyses)
    .where(and(eq(stoolAnalyses.childId, childId), sql`${stoolAnalyses.expiresAt} >= no
w()`))
    .orderBy(desc(stoolAnalyses.id))
    .limit(1)
  return row ?? null
}

export function stoolFilePath(name: string): string {
  return path.join(UPLOAD_DIR, path.basename(name))
}

export function mimeOf(filename: string): string {
  const ext = path.extname(filename).toLowerCase()
```

```
  return (
    {
      '.jpg': 'image/jpeg',
      '.jpeg': 'image/jpeg',
      '.png': 'image/png',
      '.webp': 'image/webp',
    }[ext] || 'application/octet-stream'
  )
}

export { existsSync as fileExists, createReadStream }
import fp from 'fastify-plugin'
import type { FastifyInstance } from 'fastify'

export default fp(async (fastify: FastifyInstance) => {
  fastify.addHook('preHandler', async (req) => {
    req.user = null
    const auth = req.headers.authorization
    if (auth && auth.startsWith('Bearer ')) {
      try {
        req.user = fastify.jwt.verify(auth.slice(7)) as { parent_id?: number; phone?: s
tring; role?: string }
      } catch {
        req.user = null
      }
    }
  })
})
import fp from 'fastify-plugin'
import type { FastifyInstance } from 'fastify'
import { AppError } from '../lib/appError'

export default fp(async (fastify: FastifyInstance) => {
  fastify.setErrorHandler((error, req, reply) => {
    if (error instanceof AppError) {
      reply.status(error.status).send({ code: error.code, message: error.message })
    } else if ((error as { validation?: unknown }).validation) {
      reply.status(400).send({ code: 'VALIDATION', message: '参数不正确' })
    } else {
      req.log.error(error)
      reply.status(500).send({ code: 'INTERNAL', message: '服务器开小差了，请稍后再试' })
    }
  })
})
import '@fastify/jwt'

declare module '@fastify/jwt' {
  interface FastifyJWT {
    user: { parent_id?: number; phone?: string; role?: string } | null
  }
}
```